

Domande e Risposte architettura

e# 📅 Appunti di Davide Scarlata 2024/2025



Prof: Claudio Schifanella



Mail: claudio.schifanella@unito.it



Corso: C



[Moodle Unito](#)

CIRCUITI

Cosa sono i circuiti sequenziali?

I circuiti sequenziali sono circuiti in grado di calcolare funzioni che dipendono, oltre dai dati in input, anche da uno stato interno, ovvero dipendono anche da informazioni memorizzate in elementi di memoria interni. In generale, la funzione calcolata dal circuito ad un dato istante dipende dalla sequenza temporale dei valori in input al circuito.

Differenza tra circuiti sequenziali e combinatori?

Circuiti combinatori: sono in grado di calcolare funzioni che dipendono solo dai dati in input

Circuiti sequenziali: sono in grado di calcolare funzioni che dipendono, oltre dai dati in input, anche da uno stato interno, ovvero dipendono anche da informazioni memorizzate in elementi di memoria interni.

Flip-Flop

cosa sono?

circuiti sequenziali utilizzati per memorizzare uno stato

Cosa sono i circuiti sequenziali?

I circuiti sequenziali sono circuiti in grado di calcolare funzioni che dipendono, oltre dai dati in input, anche da uno stato interno, ovvero dipendono anche da informazioni memorizzate in elementi di memoria interni. In generale, la funzione calcolata dal circuito ad un dato istante dipende dalla sequenza temporale dei valori in input al circuito.

Differenza tra circuiti sequenziali e combinatori?

Circuiti combinatori: sono in grado di calcolare funzioni che dipendono solo dai dati in input

Circuiti sequenziali: sono in grado di calcolare funzioni che dipendono, oltre dai dati in input, anche da uno stato interno, ovvero dipendono anche da informazioni memorizzate in elementi di memoria interni.

come funzionano i Flip-Flop?

hanno due ingressi e due uscite , possiedono uno stato (0 o 1).

***LO STATO** dipende dagli ingressi , dallo stato precedente e dal clock

IL CLOCK generalmente fa cambiare lo stato del flip flop ad ogni fronte di salita (quando passa da 0 a 1)

dove vengono usati? a cosa servono?

vengono raggruppati insieme per formare i **registri**. servono a memorizzare word (parole) con un numero di bit variabile (halfword, byte, word, double word, quad word)

Cosa sono DI0-DI1?

DI0, DI1, ... DIN sono le linee di ingresso dati dei Flip-Flop di un registro, ogni linea corrisponde ad un Flip-Flop.

Qual è la differenza tra Flip Flop e Latch?

1. Il fattore chiave di differenziazione tra il latch e il flip flop è che il latch cambia continuamente l'uscita in base alla variazione dell'ingresso, mentre un flip flop cambia l'uscita solo quando viene attivato l'impulso di clock insieme alla modifica dell'ingresso
2. In latch poiché l'uscita dipende solo dall'ingresso applicato, non è necessario un clock esterno. Invece, i flip flop richiedono necessariamente un segnale di clock esterno per il loro funzionamento
3. L'assenza di un clock fa sì che i latch funzionino in modo asincrono mentre, poiché il segnale di clock sincronizza il funzionamento dei flip flop, il flip flop funziona in modo sincrono
4. I latch sono attivati a livello, quindi funzionano ogni volta che l'ingresso cambia da un livello binario all'altro. I flip flop sono attivati sul fronte, quindi si attivano quando il segnale di clock passa da basso ad alto o da alto a basso.

cos'è la trasparenza di un flip flop?

La trasparenza di un flip flop indica la sua capacità di cambiare lo stato in risposta agli ingressi durante il periodo in cui il segnale di clock è attivo. In altre parole, un flip flop è trasparente quando l'uscita segue direttamente l'ingresso durante il periodo di attivazione del clock. Questo

comportamento è tipico dei latch, mentre i flip flop normalmente non sono trasparenti e cambiano stato solo al fronte di salita o discesa del clock.

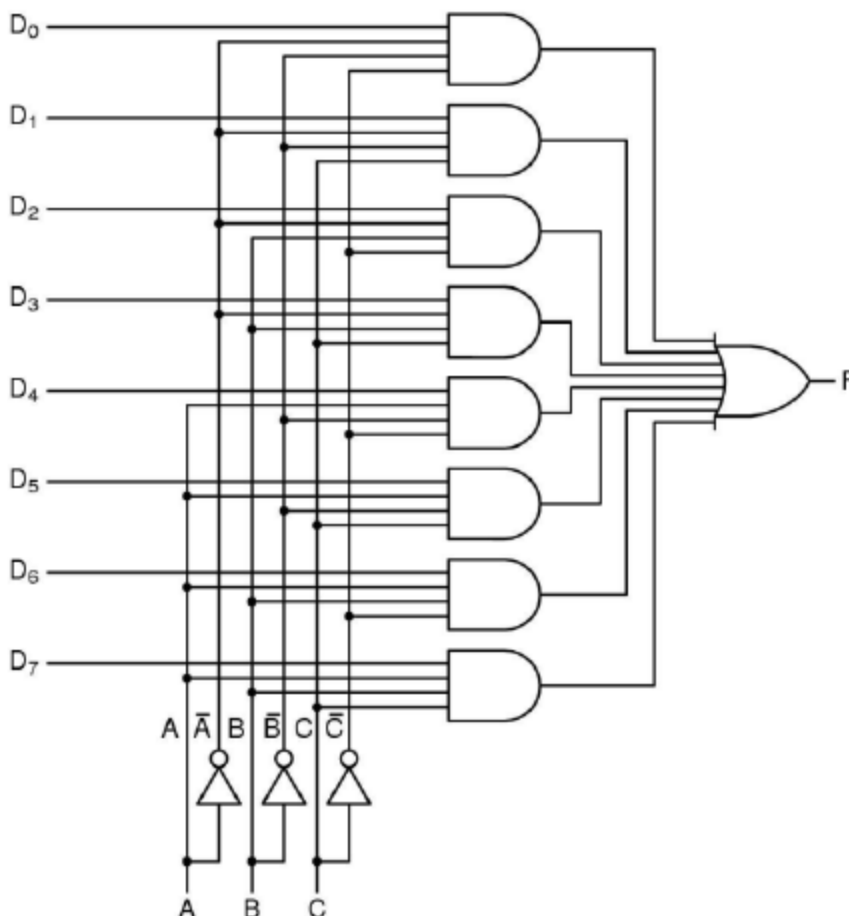
Cos'è una macchina a stati finiti? Ha circuiti sequenziali o combinatori? Da cosa dipende lo stato futuro di questa macchina?

Una macchina a stati finiti è un dispositivo automatico in grado di interagire con l'ambiente esterno. A fronte di un input, esibisce un output che dipende anche da informazioni memorizzate in elementi interni (stati). Quindi possiede al suo interno circuiti sequenziali. Lo stato futuro di una macchina a stati finiti dipende dall'input e dallo stato attuale.

S: $A \times Q \rightarrow Q$ funzione di cambiamento di stato A = simbolo in ingresso, Q = simbolo di stato

Disegnare e spiegare il multiplexer

Il multiplexer è un circuito combinatorio che, in generale, possiede 2^n ingressi, 1 uscita e n ingressi di controllo. Le linee di controllo determinano quale dei 2^n ingressi deve essere selezionato per essere inviato all'uscita. Esempio con 8 ingressi ($D_0, \dots, D_7$), un'uscita (F) e tre ingressi di controllo (A, B e C):



Il multiplexer è sequenziale o combinatorio?

Il multiplexer è un circuito combinatorio

Spiegare e disegnare half adder e full adder?

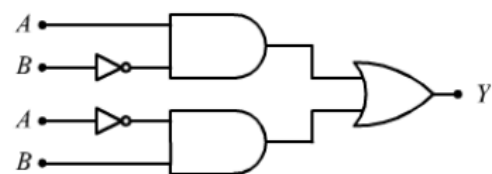
I sommatore binari sono reti combinatorie che ricevono in ingresso n bit degli addendi da sommare e generano in uscita i bit della somma binaria con il relativo riporto.

$$Y = A \oplus B = A\bar{B} + \bar{A}B$$



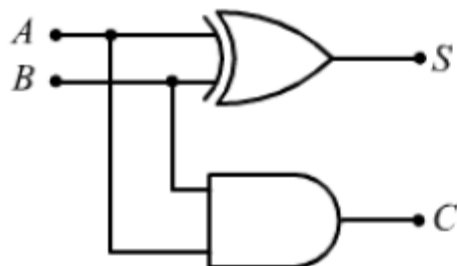
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$ con riporto 1

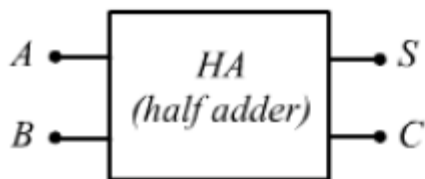


Questo schema non è altro che la tabella di verità della porta XOR (a parte la questione del riporto che deve essere risolto).

Per gestire anche il riporto questo circuito deve essere modificato in modo da risolvere la seguente tabella della verità:



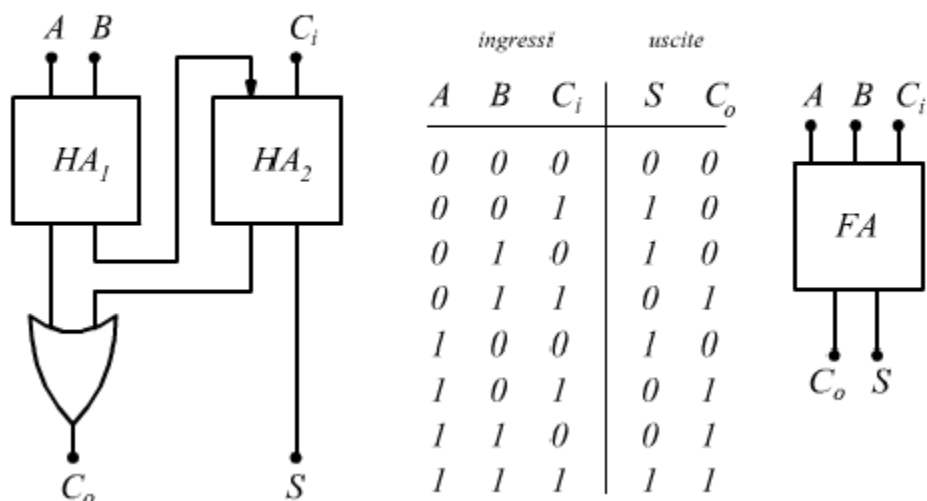
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



$$S = A \oplus B$$
$$C = A \cdot B$$

A e B sono i bit da sommare, S è il bit della somma (sum) e C è il bit del riporto (carry).

Full-Adder



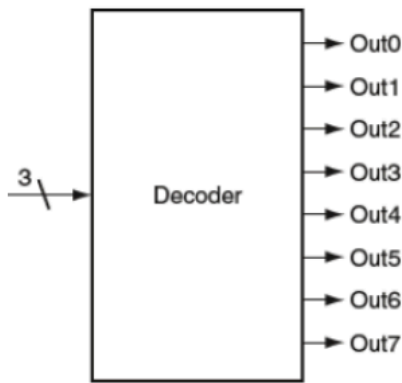
Per ottenere la somma completa tra due numeri di più cifre, oltre ai bit dello stesso ordine occorre sommare anche il riporto eventualmente ottenuto dai due bit di ordine immediatamente inferiore, per questo motivo il circuito full-adder si presenta con tre ingressi e due uscite.

Cos'è il decoder e come funziona?

Il decoder è un circuito combinatorio che prende un numero di n bit come ingresso e lo usa per selezionare (mettere a 1, asserire) una delle 2^n linee di uscita. Può essere utilizzato per attivare una certa componente, un banco di memoria, ecc.

I due ingressi sono costituiti dai due bit A_n e B_n da sommarsi e dal riporto C_i eventualmente ottenuto dalla somma dei due bit A_{n-1} e B_{n-1} .

Le due uscite sono composte dal bit di somma dei tre ingressi e dall'eventuale riporto C_o da inviare al full-adder successivo.

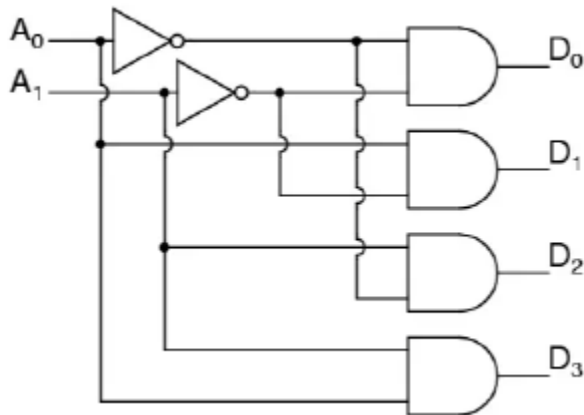


a. A 3-bit decoder

Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

Gli ingressi vanno interpretati come le cifre di un numero in base 2 con 12 che è la cifra più significativa.



Esempio di decoder a 2 bit con 4 uscite

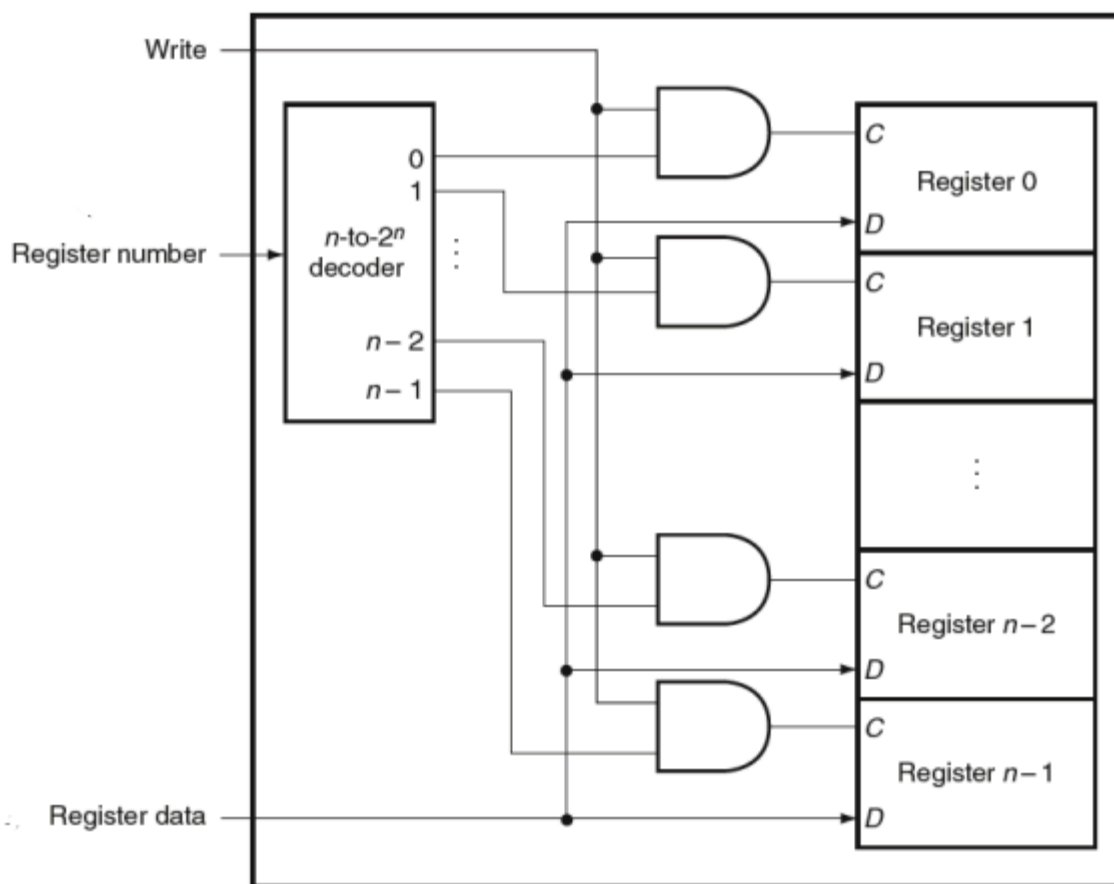
Come collabora il decoder con il blocco dei registri?

Nel blocco dei registri, il decoder viene utilizzato per stabilire su quale registro scrivere quando il segnale di Write è abilitato.

Ha come input il numero del registro in binario e l'uscita di output è collegata al registro

corrispondente.

Porta di scrittura



Memorie

Spiegare gerarchia della memoria

La gerarchia della memoria è ordinata secondo la **capacità**, i **costi di memorizzazione** e i **tempi di accesso all'informazione**. È strutturata come una piramide: più in **alto** troviamo le **memorie più piccole e più costose**, ma il cui tempo di accesso all'informazione è minore, **più in basso** troviamo le **memorie più capienti e meno costose**, ma il cui tempo di accesso all'informazione è maggiore.

- Registri
- Cache
- RAM
- SSD
- HDD (disco rigido)

Cos'è un blocco di memoria?

Un blocco di memoria è un insieme di celle di memoria che possono essere lette o scritte in modo atomico, ovvero tutte le celle vengono lette o scritte contemporaneamente. Un blocco di memoria può essere costituito da più celle di memoria, ad esempio un byte è un blocco di memoria composto da 8 bit.

Differenza tra blocco e linea

Un blocco di memoria è un insieme di celle di memoria che possono essere lette o scritte in modo atomico, mentre una linea di memoria è una singola cella di memoria. Un blocco di memoria può essere composto da più linee di memoria, ad esempio un blocco di 4 byte è composto da 4 linee di memoria.

Come sono fatti i diversi tipi di memoria? Anche nello specifico i diversi tipi di RAM

Ci sono due tipi principali di RAM:

- RAM Statica (SRAM)
- RAM Dinamica (DRAM)

La RAM Statica (SRAM) è composta da celle di memoria che utilizzano flip-flop per memorizzare i dati, il che la rende più veloce ma anche più costosa e con una densità di memorizzazione inferiore rispetto alla DRAM (perchè usano più transistor) . La SRAM mantiene i dati finché l'alimentazione è attiva.

La RAM Dinamica (DRAM) utilizza condensatori per memorizzare i dati, il che la rende più lenta ma anche meno costosa e con una densità di memorizzazione superiore rispetto alla SRAM. La DRAM deve essere continuamente aggiornata per mantenere i dati, poiché i condensatori si scaricano nel tempo.

Altri tipi di memoria: memorie flash, memorie a disco

Le memorie flash sono un tipo di memoria programmabile a sola lettura, cancellabile elettricamente. A differenza dei dischi e delle DRAM, i bit delle memorie flash si deteriorano dopo un certo numero di scritture.

Le memorie a disco sono formate da un gruppo di dischi, detti piatti, che ruotano ad una predeterminata velocità. Ciascun piatto metallico è ricoperto su entrambe le facce da materiale magnetico registrabile. Per leggere e scrivere dati su un disco, al di sopra di ognuna delle due superfici di ogni disco è posizionato un braccio mobile contenente una piccola bobina elettromagnetica chiamata testina di lettura/scrittura. La superficie di ogni disco è divisa in cerchi concentrici chiamati tracce, ogni traccia è a sua volta suddivisa in settori, che sono l'unità di memorizzazione delle informazioni. Per accedere ai dati, il sistema operativo effettua un'operazione chiamata seek, e il tempo necessario per spostare la testina sopra la traccia

desiderata viene chiamato tempo di ricerca (seek time). Quando la testina di lettura/scrittura ha raggiunto la traccia corretta, deve aspettare che il settore desiderato passi sotto. Questo tempo di attesa viene chiamato latenza di rotazione o ritardo di rotazione.

REGISTRI

Spiegare il funzionamento ed il ruolo dei vari tipi di registri?

RISC-V possiede 32 registri per gli interi, 32 registri per i numeri in virgola mobile, un Program Counter (PC) e 4096 Control Status Register. I ruoli dei 32 registri per gli interi sono così suddivisi:

- **32 registri interi** ($x0$ - $x31$)
- **32 registri floating-point** ($f0$ - $f31$)
- ⚙ **Control Status Register** (gestione stato CPU)
- 🎯 **Program Counter (PC)** (indica l'istruzione successiva)

Registro	Nome	Utilizzo
x0	zero	Costante zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Puntatore a thread
x8	s0 / fp	Frame pointer (il contenuto va preservato se utilizzato dalla procedura chiamata)
x10-x11	a0-a1	Passaggio di parametri nelle procedure e valori di ritorno
x12-x17	a2-a7	Passaggio di parametri nelle procedure
x5-x7 x28-x31	t0-t2 t3-t6	Registri temporanei, non salvati in caso di chiamata
x9	s1	Registri da salvare: il contenuto va preservato se utilizzati dalla procedura chiamata
x18-x27	s2-s11	

La ALU può leggere e scrivere solo dai registri, l'accesso alla memoria è più lento rispetto a quello dei registri, il compilatore si occupa di individuare la strategia più efficiente per le operazioni di caricamento e salvataggio dei dati tra registri e memoria, variabili utilizzate più di frequente devono rimanere il più possibile salvate nei registri.

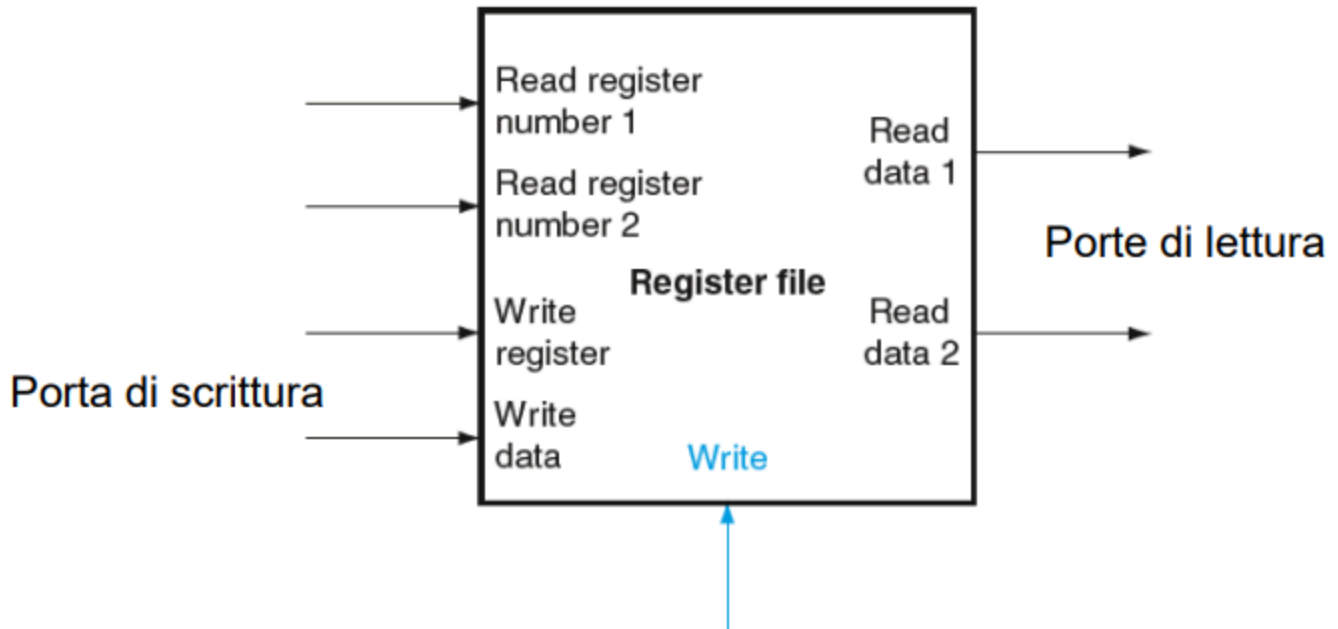
Come funziona il blocco dei registri (Register File)?

Il blocco di registri è un insieme di registri leggibili e scrivibili.

Si deve fornire in ingresso il numero del registro sul quale compiere l'operazione.

Si implementa con un multiplexer per ogni porta di lettura, con un decoder per la porta di

scrittura ed un insieme di registri basati su flip-flop di tipo D.



Come si recupera qualcosa dalla memoria?

Significa **leggere un valore** (es. un numero) **che si trova in RAM**, e copiarlo in un **registro** della CPU per poterlo usare (es. sommare, confrontare, stampare...).

Nel linguaggio RISC-V, questo si fa con le **istruzioni di tipo load** (caricamento).

Cosa serve per accedere alla memoria in RISC-V?

1. **Un indirizzo di memoria** → cioè "dove" si trova il dato.
2. **Un'istruzione load** → per leggere il dato.
3. **Un registro di destinazione** → dove salvare il dato letto.

Cosa succede internamente nella CPU

1. **Calcolo dell'indirizzo:** $\text{indirizzo} = \text{contenuto di } x5 + \text{offset}$
Es: $0x1000 + 16 = 0x1010$
2. **La CPU invia questo indirizzo** alla memoria (tramite il **bus degli indirizzi**).
3. **Attiva la linea di controllo "READ"** (lettura).
4. **Aspetta:** la memoria impiega qualche nanosecondo a rispondere (tempo di accesso).
5. **La memoria invia 8 byte** all'indirizzo $0x1010$ sul **bus dati**.
6. **La CPU riceve questi dati** e li **salva in** $x10$.

Componenti coinvolti

 **Dentro la CPU:**

- **ALU (Arithmetic Logic Unit)** → calcola l'indirizzo effettivo.
- **Register File** → contiene i registri (x0-x31).
- **Control Unit** → decodifica l'istruzione.
- **Load/Store Unit** → gestisce gli accessi alla memoria.

Fuori dalla CPU:

- **Bus indirizzi** → invia l'indirizzo alla RAM.
- **Bus dati** → riceve il valore letto.
- **RAM (Memoria principale)** → contiene i dati e le istruzioni.

Fasi del recupero dalla memoria (es: `ld x10, 16(x5)`)

1. Fetch dell'istruzione

- Il **Program Counter (PC)** punta all'indirizzo dell'istruzione `ld x10, 16(x5)`.
- L'**istruzione** viene letta dalla memoria **istruzioni** e **inviata alla CPU**.

2. Decodifica

- La **Control Unit** riconosce il tipo di istruzione: `ld` (load doubleword).
- Capisce:
 - **Registro di destinazione:** `x10`
 - **Registro base:** `x5`
 - **Offset:** `16`

3. Lettura dei registri

- Il **Register File** fornisce il valore contenuto in `x5` (es: `0x1000`).
- Offset è immediato → già disponibile.

4. Calcolo indirizzo effettivo

- La **ALU** calcola:
`indirizzo effettivo = x5 + offset = 0x1000 + 16 = 0x1010`

5. Accesso alla memoria

- Il risultato `0x1010` viene inviato alla **Load/Store Unit**.
- La **Load Unit**:
 - Mette `0x1010` sul **bus degli indirizzi**.
 - Attiva la linea di controllo per la **lettura** (Read Enable).

6. Lettura dalla RAM

- La memoria riceve la richiesta:
 - “Fornisci 8 byte a partire dall’indirizzo `0x1010`”.
- Dopo un **tempo di accesso** (latenza), la RAM invia il dato sul **bus dati**.

7. Scrittura nel registro

- La CPU riceve i dati (8 byte).
- Il dato viene **inviato al Register File** e scritto nel **registro** `x10`.

La Cache

Cos’è la cache?

La cache è un’area di memoria estremamente veloce ma con una capacità molto bassa. Il suo scopo è quello di velocizzare l’esecuzione dei programmi conservando le parole di memoria più utilizzate da essi. L’uso della memoria cache è basato sul principio di località del codice.

Come decidiamo cosa mettere nella cache: località temporale e spaziale?

L’uso della memoria di cache è basato sul principio di località del codice: i programmi non accedono alla memoria a caso. Le parole di memoria più usate vengono tenute in una cache.

Località temporale: principio secondo cui se si accede ad una determinata locazione di memoria, è molto probabile che vi si acceda di nuovo dopo poco tempo.

(es. variabili usate in un ciclo)

Località spaziale: principio secondo cui se si accede ad una determinata locazione di memoria, è molto probabile che si acceda alle locazioni vicine ad essa dopo poco tempo.

(es. array, strutture dati)

Le istruzioni sono salvate in cache?

Sì, è possibile che le istruzioni vengano salvate nella cache per velocizzare l’esecuzione del programma. La cache può contenere sia dati che istruzioni, a seconda di come è configurata.

Descrivere i 3 tipi di cache? Come funzionano i principi di località temporale e spaziale nei 3 tipi?

tre tipi di cache sono:

- Mappatura diretta
- Completamente associativa
- Set-associativa

Componente	Funzione
Linea di cache	Un blocco (ad es. da 1 parola o più parole)
Indice	Stabilisce quale linea di cache usare
Tag	Verifica se il dato cercato è proprio quello memorizzato
Bit di validità	Indica se la linea contiene un dato valido oppure no

Mappatura diretta

1. Mappatura diretta ●

Come funziona:

- Ogni **blocco di memoria principale** può andare in **un solo blocco** della cache.
- Si usa una parte dell'indirizzo come **indice** nella cache.

Indirizzamento e calcolo dell'indice

Se:

- La cache ha **N** linee
- L'indirizzo di memoria si riferisce al blocco **B**

Allora l'indice della cache sarà:

$$\text{Indice} = B \bmod N$$

Come si capisce se il dato è presente?

1. Bit di validità

- Dice se quella linea **contiene un dato valido o è vuota/sporca**
- Serve per sapere se possiamo **fidarci** di ciò che c'è nella riga

2. TAG

- Parte **superiore dell'indirizzo** usata per distinguere tra blocchi che finiscono nello stesso indice
- Se **indice e tag** corrispondono, e **valid = 1** → **HIT**
- Altrimenti → **MISS**

+ Vantaggi:

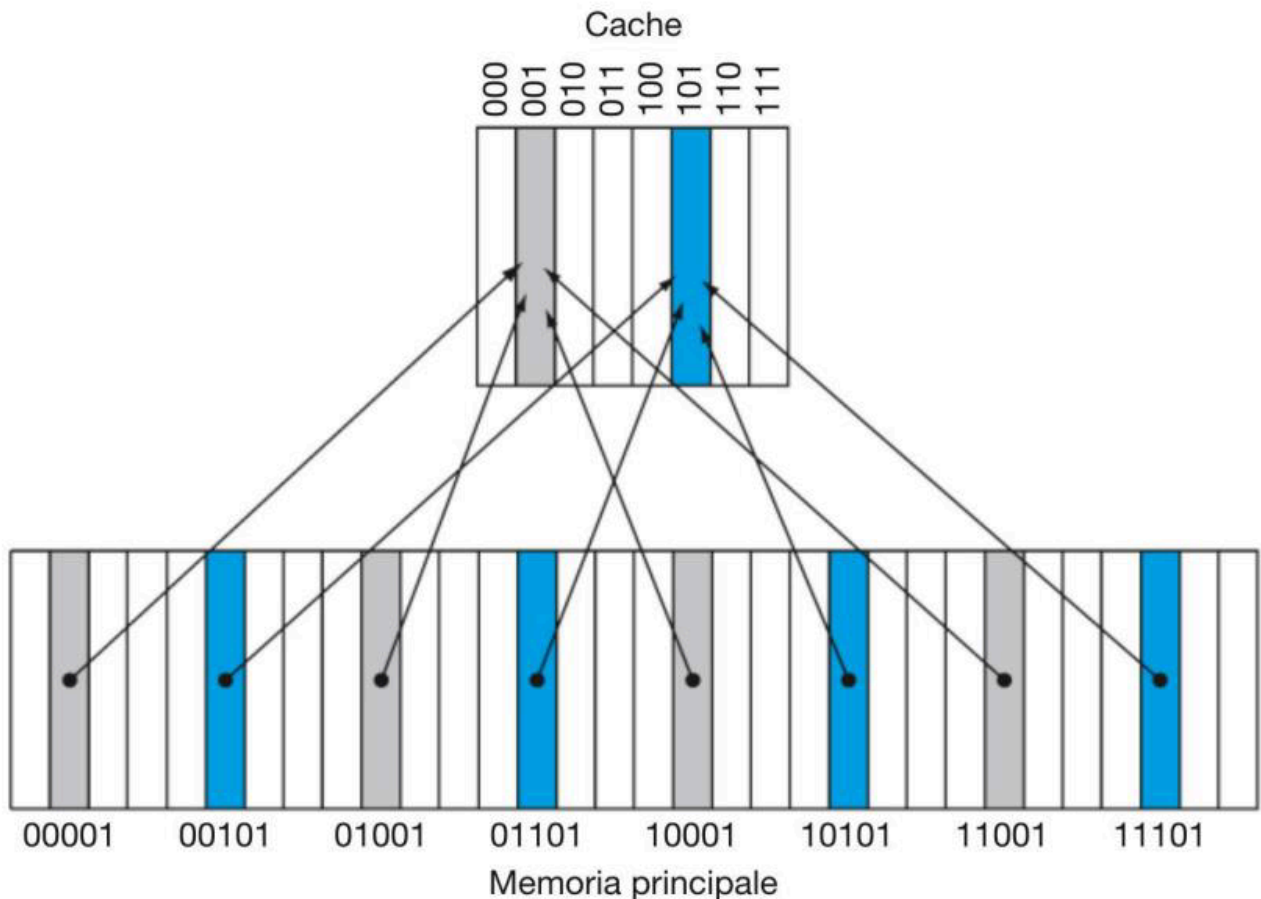
- ☒ Molto semplice da implementare
- ☒ Veloce (1 confronto)

— Svantaggi:

- ☒ Alta possibilità di **collisioni**: due dati diversi che usano lo stesso indice si sovrascrivono continuamente (conflitti)

🧠 Località:

- **Spaziale**: buona, se i dati usano indirizzi vicini
- **Temporale**: rischia di essere compromessa se più dati "si combattono" lo stesso posto in cache



Completamente associativa

Definizione:

Ogni **blocco della memoria principale può essere caricato in** qualsiasi riga (linea)** della cache.

Caratteristica	Descrizione
Flessibilità totale	Nessun vincolo sull'indice: un blocco può andare in qualsiasi posizione
Nessun calcolo di indice	L'indirizzo di memoria non viene suddiviso in tag/indice/offset come nel caso diretto
Ricerca parallela	Tutte le righe della cache vengono confrontate in parallelo tramite comparatori
	Per capire se un dato è presente, si confrontano tutti i tag

Come si verifica se un dato è presente

1. Si confronta il **tag dell'indirizzo** con **tutti i tag salvati** nella cache
2. La ricerca è **parallela** (hardware complesso)
3. Se uno dei tag corrisponde e la riga è valida → **HIT**
4. Altrimenti → **MISS**

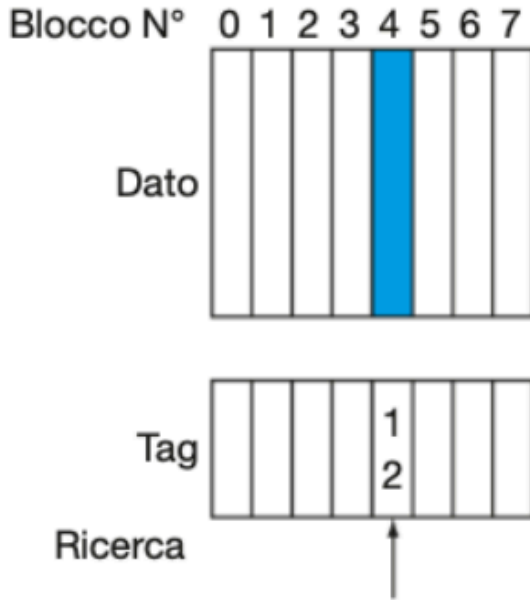
Svantaggi

Aspetto	Problema
Costo hardware	Ogni riga deve avere un comparatore → più lenta e costosa
Scalabilità limitata	Adatta solo a cache piccole , altrimenti il numero di confronti esplode

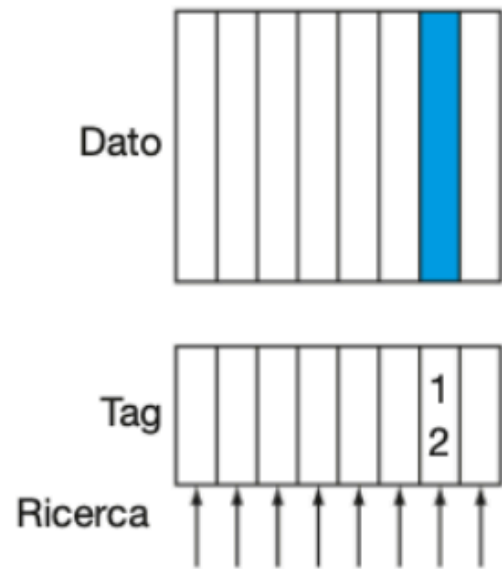
Vantaggi

Aspetto	Vantaggio
Zero conflitti	Due blocchi non si combattono mai la stessa riga
Massima flessibilità	La cache può decidere dove salvare un dato per ottimizzare lo spazio

Mappatura diretta



Completamente associativa



Set-associativa

Definizione:

La cache è divisa in **set**, ciascun set contiene più **linee (blocchi)** chiamate anche "way". Ogni blocco di memoria può essere mappato in **un solo set**, ma in quel set può occupare **qualunque linea** (way).

Termine	Significato
Set	Gruppo di k linee (es. 2-way, 4-way significa 2 o 4 linee per set)
Linee (way)	Blocchi all'interno di un set
Indice	Indica il set (anziché la singola linea come nella diretta)
Tag	Identifica il blocco di memoria
Bit validità	Indica se la linea è valida

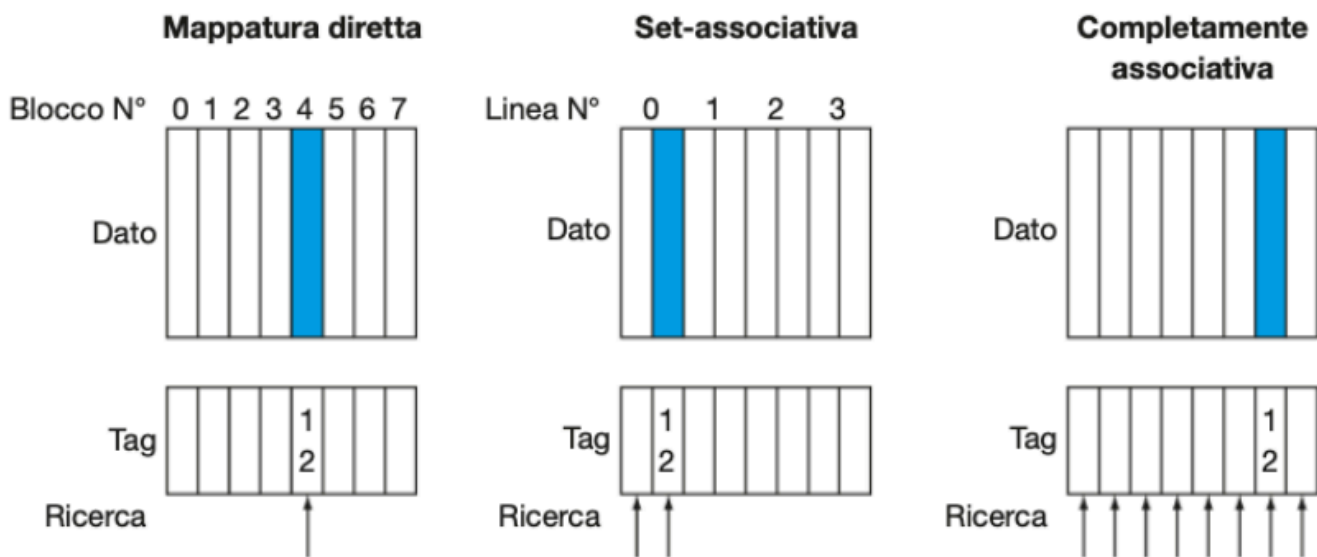
1234 Come si calcola l'indice del set?

Dato un blocco B e un numero totale di set S :
indice del set = B mod S

Come funziona la ricerca?

1. Calcola l'indice del set usando l'indirizzo.
2. Vai al set corrispondente, che ha k linee (way).
3. Confronta il **tag** di tutti i k blocchi nel set **in parallelo**.
4. Se uno dei tag corrisponde e il bit validità è attivo → **HIT**.
5. Altrimenti → **MISS**.

Aspetto	Dettaglio
Compromesso tra diretto e associativo	Meno conflitti rispetto alla mappatura diretta, meno costosa della completamente associativa
Flessibilità	Ogni blocco ha più possibilità di posizionamento nel suo set
Prestazioni	Più veloce della completamente associativa, migliore della diretta per accessi contesi
Hardware	Necessita di k comparatori (per i k way)



Cosa cambia il numero di linee all'interno della cache?

Il numero di linee all'interno della cache determina la quantità di dati che possono essere memorizzati nella cache e la velocità con cui i dati possono essere recuperati. Un numero maggiore di linee consente una maggiore capacità di memorizzazione e una migliore gestione della località temporale e spaziale, ma può anche aumentare la complessità del circuito e i costi associati alla cache.

perchè la cache porta un vantaggio nell' esecuzione di un programma ?

La cache porta un vantaggio nell'esecuzione di un programma perché consente di memorizzare temporaneamente i dati e le istruzioni più frequentemente utilizzati, riducendo il tempo di accesso alla memoria principale. Questo sfrutta i principi di località temporale e spaziale, permettendo alla CPU di accedere rapidamente ai dati necessari senza dover attendere il caricamento dalla memoria principale, che è più lenta. Di conseguenza, la cache migliora le prestazioni complessive del sistema e riduce i tempi di esecuzione dei programmi.

strategie per modificare il valore nella cache:

Scrittura attraverso (write-through)

In questo metodo, quando un dato viene scritto nella cache, viene immediatamente scritto anche nella memoria principale. Questo garantisce che la memoria principale sia sempre aggiornata, ma può rallentare le operazioni di scrittura poiché richiede due operazioni di scrittura.

Scrittura differita (write-back)

In questo metodo, i dati vengono scritti nella cache e solo successivamente, quando il blocco viene rimosso dalla cache, vengono scritti nella memoria principale. Questo riduce il numero di operazioni di scrittura sulla memoria principale, migliorando le prestazioni, ma richiede una gestione più complessa per garantire che i dati nella memoria principale siano aggiornati correttamente.

Scrittura per aggiornamento (write-update)

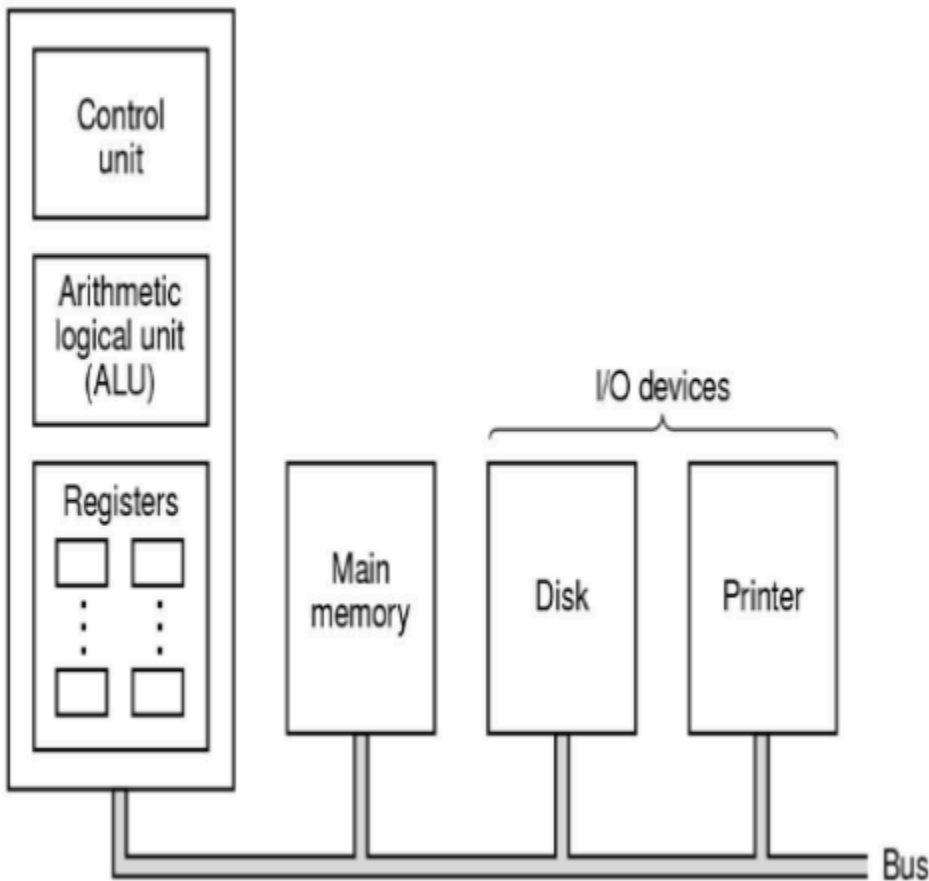
In questo metodo, quando un dato viene scritto nella cache, viene aggiornato anche nella memoria principale, ma solo se il blocco è già presente nella cache. Se il blocco non è presente, la scrittura viene ignorata. Questo metodo può essere utile per ridurre il numero di operazioni di scrittura sulla memoria principale, ma può comportare una maggiore complessità nella gestione della cache.

macchina di von Neumann

Spiegare la macchina di Von Neumann

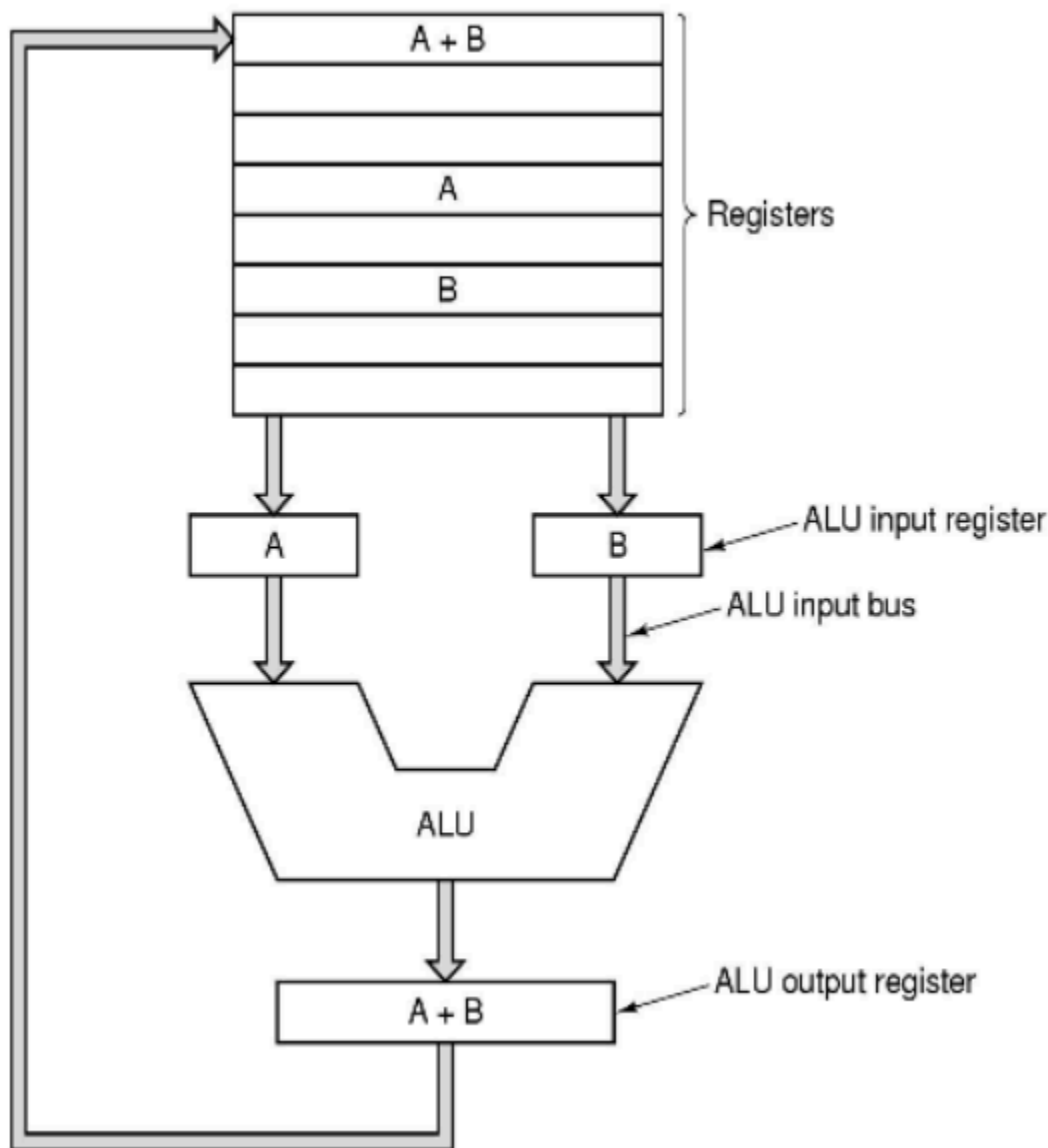
La macchina di Von Neumann si basa su 4 componenti fondamentali: CPU, RAM, Interfaccia di I/O, Bus di sistema.

Central processing unit (CPU)



La CPU si compone di diverse parti distinte: Control Unit, ALU, registri. I registri, l'ALU e alcuni bus che li collegano compongono il data path. La main memory contiene sia istruzioni, sia dati, usando sequenze di bit.

Com'è strutturata la CPU e come funziona?



Data path: organizzazione interna di una CPU (registri, ALU, bus interno).

Registro: memoria veloce per dati temporanei. Istruzioni registro-registro e registro-memoria.

Ciclo del data path: processo di far passare due operandi attraverso l'ALU e memorizzarne il risultato.

Ciclo di fetch-decode-execute La CPU esegue ogni istruzione del livello 1 (ISA) per mezzo di una serie di passi elementari:

1. Prendi l'istruzione seguente dalla memoria e mettila nel registro delle istruzioni
2. Cambia il program counter per indicare l'istruzione seguente
3. Determina il tipo dell'istruzione appena letta
4. Se l'istruzione usa una parola in memoria, determina dove si trova
5. Metti la parola, se necessario, in un registro della CPU
6. Esegui l'istruzione
7. Torna al punto 1 e inizia a eseguire l'istruzione successiva

Come funziona l'unità di controllo? Com'è fatta dentro e quali sono i vari passaggi interni per selezionare la funzione richiesta?

L'unità di controllo è la parte della CPU che contiene i circuiti che effettivamente decodificano le istruzioni.

Ha come input i 7 bit dell'istruzione (codop) e ha diversi output (totale 8 bit):

- Due segnali a 1 bit utilizzati per controllare i multiplexer (ALUSrc e MemtoReg)
- Tre segnali a 1 bit per controllare lettura e scrittura del register file e della memoria dati (RegWrite, MemRead e MemWrite)
- Un segnale a 1 bit utilizzato come segnale di controllo per i salti condizionati (Branch)
- Un segnale di controllo a 2 bit per la ALU (ALUOp)

Codice operativo (codop): il campo che denota il tipo di operazione e il formato di un'istruzione. A seconda del codop, il campo funz3 e funz7 servono come campi di estensione del codice operativo. AluOp: bit utilizzati come ingressi per selezionare l'operazione che la ALU deve effettuare. AluOp + Funz = Controllo della ALU.

Valori delle linee di controllo:

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Effetto di ciascuno dei restanti sei segnali di controllo

Non asserito = 0; Asserito = 1

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nulla	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di PC + 4	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nulla	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nulla	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

[illegible]

La CPU comunica con i device di I/O tramite i controllori, che hanno il compito di trasformare:

- Ogni controllore ha al suo interno dei registri identificati da un indirizzo, che può:

- La comunicazione con i controllori è simile agli accessi in memoria

- Dati (ingresso e uscita): esempio, per una stampante i dati da stampare

- Busy waiting: La CPU controlla lo stato di avanzamento dell'I/O e lo stato della periferica ispezionando in un ciclo il bit del registro di stato del controllore READY fino a che questi non segnala di essere pronto per un nuovo comando di I/O.

Svantaggio: la CPU rimane in ciclo attendendo che il dispositivo sia pronto, da qui il nome busy waiting.

Scrittura es. display con busy waiting :

- la CPU attende che il dispositivo sia pronto ad accogliere un carattere (bit Ready = 1)
- la CPU invia il carattere sul display buffer
- il controllore azzerà il bit Ready e visualizza il contenuto del buffer
- il controllore segnala di essere pronta a ricevere un nuovo comando (bit Ready = 1)

Polling: verifica ciclica (periodica) dei dispositivi di I/O tramite i bit di controllo.

Interrupt: permette di liberarsi dal busy waiting, il dispositivo quando ha finito genera un segnale che avverte la CPU di aver completato il proprio lavoro (interrupt). La CPU può abilitare l'interrupt mettendo a 1 un opportuno bit. Gli interrupt sono asincroni rispetto al programma e devono essere gestiti in modo trasparente, ovvero lo stato dell'esecuzione dopo la gestione dell'interrupt deve tornare esattamente come era prima dell'interrupt stesso.

DMA (Direct Memory Access): permette di trasferire dati tra un dispositivo di I/O e la memoria senza l'intervento della CPU. La CPU invia un comando al controller DMA, che poi gestisce il trasferimento autonomamente. Questo libera la CPU da operazioni di I/O e migliora le prestazioni complessive del sistema.

Spiegare le varie fasi del passaggio dal codice C al linguaggio macchina

◆ 1. Compilatore

- **Input:** codice C
 - **Output:** codice **assembly** (livello 3)
 - Traduce istruzioni C in **mnemonici ISA** (come `addi`, `lw`, `sw`, ecc.)
-

◆ 2. Assemblatore (Assembler)

- **Input:** codice assembly (livello 3)
- **Output:** **file oggetto (.o)** in linguaggio macchina (livello 1)
- trasforma l'assembly in **codice binario** (livello 1)

Assemblatore a due passi:

Passo	Azione
1	Raccoglie tutte le etichette (label), crea una symbol table
2	Traduce le istruzioni assembly in istruzioni binarie e risolve le etichette usando la tabella

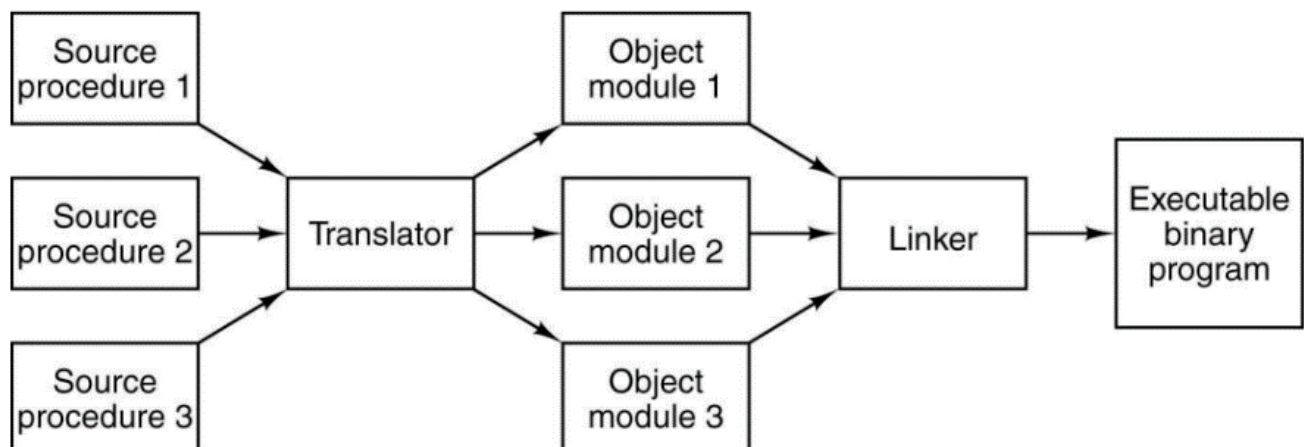
◆ 3. Linker

- **Input:** moduli oggetto (compilati e assemblati separatamente)
- **Output:** eseguibile unico

Cosa fa il linker:

1. Unisce i moduli oggetto in un solo **spazio di indirizzamento**
2. Costruisce una **tabella dei moduli** e delle loro lunghezze
3. Assegna un **indirizzo iniziale a ogni modulo**
4. **Relocation:** aggiusta gli indirizzi interni per renderli coerenti nello spazio unico
5. Risolve **riferimenti a funzioni esterne** (es. da librerie)

Modulo	Lunghezza	Indirizzo partenza
A	400	100
B	600	500
C	500	1100
D	300	1600



◆ 4. Loader

- **Input:** file eseguibile (da disco)
- **Output:** codice caricato in **memoria RAM**, pronto per essere eseguito

Compiti del loader:

1. Legge intestazione del file (es. ELF) → determina **dimensione** e **segmenti**
2. Alloca spazio in memoria
3. Copia codice e dati nella **RAM**
4. Alloca lo **stack** e copia eventuali **parametri**
5. Inizializza **registri**, incluso lo **stack pointer**
6. Salta alla funzione `main()` per **avviare l'esecuzione**

Quali sono le differenze tra linking statico e dinamico? Quali sono i vantaggi e svantaggi di ognuno?

Cos'è il Linking Statico?

Nel **linking statico**, il linker copia **tutto il codice delle librerie usate direttamente dentro l'eseguibile**.

👉 Una volta creato, il file `.exe` o `.out` **contiene tutto ciò che serve**.

✅ Vantaggi:

- **Indipendenza:** non servono librerie esterne per eseguire il programma.
- **Portabilità:** funziona su qualsiasi computer, anche se le librerie non sono installate.
- **Avvio più rapido:** non deve cercare librerie esterne.
- **Stabilità:** se tutto funziona oggi, funzionerà anche domani (nessuna libreria esterna che può cambiare).

❌ Svantaggi:

- **File più grandi:** include codice che magari viene usato poco.
- **Meno flessibilità:** se esce una nuova versione della libreria, **devi ricompilare** il programma per usarla.
- **Maggior uso di memoria:** ogni programma ha la sua copia della libreria.

🔄 3. Cos'è il Linking Dinamico?

Nel **linking dinamico**, il linker **non copia il codice della libreria**, ma **inserisce solo un riferimento** a essa.

👉 Il codice della libreria verrà **caricato solo a runtime (quando serve)**, da file come `.so` (Linux) o `.dll` (Windows).

✅ Vantaggi:

- **File più piccoli:** non contengono tutto il codice delle librerie.
- **Memoria condivisa:** più programmi possono usare **la stessa libreria caricata una sola volta in RAM**.
- **Aggiornabilità:** se una libreria migliora o viene corretta, tutti i programmi che la usano **ne beneficiano subito**.
- **Manutenzione più facile.**

❌ Svantaggi:

- **Dipendenza da librerie esterne:** se manca una libreria o ne hai una versione sbagliata → crash.
- **Possibili conflitti:** due programmi potrebbero richiedere versioni diverse della stessa libreria.
- **Avvio più lento:** il sistema deve cercare e caricare le librerie a runtime.

Cosa significa invocare il main?

Il main generalmente è la prima funzione eseguita durante l'avvio di un programma. Invocare il main significa avviare l'esecuzione di esso, a seguito dell'operazione di loading del programma in memoria da parte del sistema operativo

i bus

Quali sono i tipi di bus?

🧱 I tre tipi fondamentali di bus

1. 🕒 Bus degli indirizzi (Address Bus)

- **Funzione:** specifica *dove* leggere o scrivere in memoria o nelle periferiche.
- **Unidirezionale:** va dalla CPU verso la memoria o i dispositivi.
- **Lunghezza:** determina **quanta memoria si può indirizzare**:
 - Es. con 32 linee → $2^{32} = 4$ GB indirizzabili

2. 📦 Bus dei dati (Data Bus)

- **Funzione:** trasporta i **dati veri e propri** tra CPU, RAM, dispositivi I/O.

- **Bidirezionale:**
 - CPU può leggere dati dalla RAM (input)
 - CPU può scrivere dati nella RAM (output)
- **Larghezza:** indica **quanti bit vengono trasferiti alla volta**
 - Es. 32 bit = trasferisce 4 byte per volta

3. 🎮 **Bus di controllo (Control Bus)**

- **Funzione:** trasmette **segnali di comando e stato** che coordinano il funzionamento del sistema.
- **Esempi di segnali:**
 - Read / Write
 - Clock
 - Reset
 - Interrupt request (IRQ)
 - Memory vs I/O select



Impatto della larghezza dei bus

Tipo di Bus	Ruolo	Impatto Architettureale
Data Bus	Trasporta i dati (in e out)	Se largo 64 bit, trasferisce 8 byte alla volta
Address Bus	Seleziona locazioni di memoria	Un bus a 32 bit → 4 GB indirizzabili
Control Bus	Governa le operazioni di lettura/scrittura	Serve per sincronizzare e controllare l'accesso ai dispositivi

Bus “larghi” sono più costosi di quelli stretti ma offrono una banda più larga e quindi maggiore velocità di trasferimento. Maggiore velocità si ottiene aumentando la larghezza del bus (più bit/trasferimento) o diminuendo il ciclo di bus (più trasferimenti/secondo).

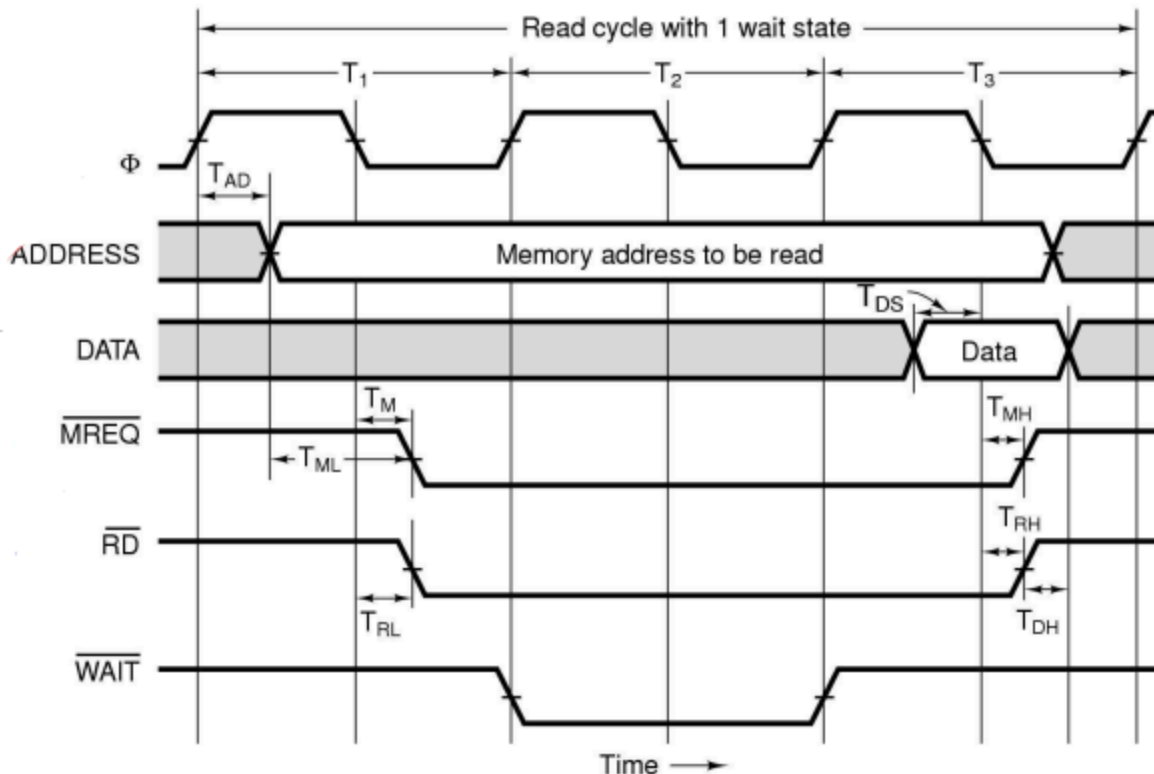
Per ovviare ai problemi dati da bus molto larghi talvolta si opta per un multiplexed bus: le linee utilizzate per il trasferimento dei dati e degli indirizzi sono le stesse; prima si trasmettono gli indirizzi e poi i dati. Ovviamente il bus multiplexato è più lento.

I bus si possono dividere in due categorie ben distinte:

Bus sincroni

hanno una linea pilotata da un oscillatore, che ha una determinata frequenza. Tutte le attività del bus richiedono un numero intero di questi cicli.

Bus sincrono – ciclo di lettura



Bus Sincrono – Ciclo di Lettura

Tempistiche e Segnali

- **Tempistiche note:** La durata delle fasi è prefissata e nota sia alla CPU che alla memoria.
- **Incertezza iniziale:** L'unico elemento variabile è l'inizio del ciclo, indicato dalla linea **MREQ**.

Fasi del Ciclo di Lettura

1. CPU genera indirizzo

- Dopo un tempo di stabilizzazione (**TAD**), la CPU pone l'indirizzo sul **bus indirizzi**.
- L'indirizzo resta stabile per tutta la durata necessaria alla lettura.

2. Richiesta di lettura

- A indirizzo stabilizzato, la CPU attiva:
 - **MREQ**: Richiesta di accesso alla memoria.
 - **RD**: Richiesta di lettura.

3. Eventuale attesa

- All'inizio del ciclo **T2**, se i dati non sono ancora pronti, la memoria può attivare **WAIT** per segnalare alla CPU di attendere.

4. Trasferimento dati

- La memoria mette i dati sul **bus dati**.

5. Conclusione del ciclo

- Una volta che i dati sono letti, la CPU disattiva **MREQ** e **RD**.

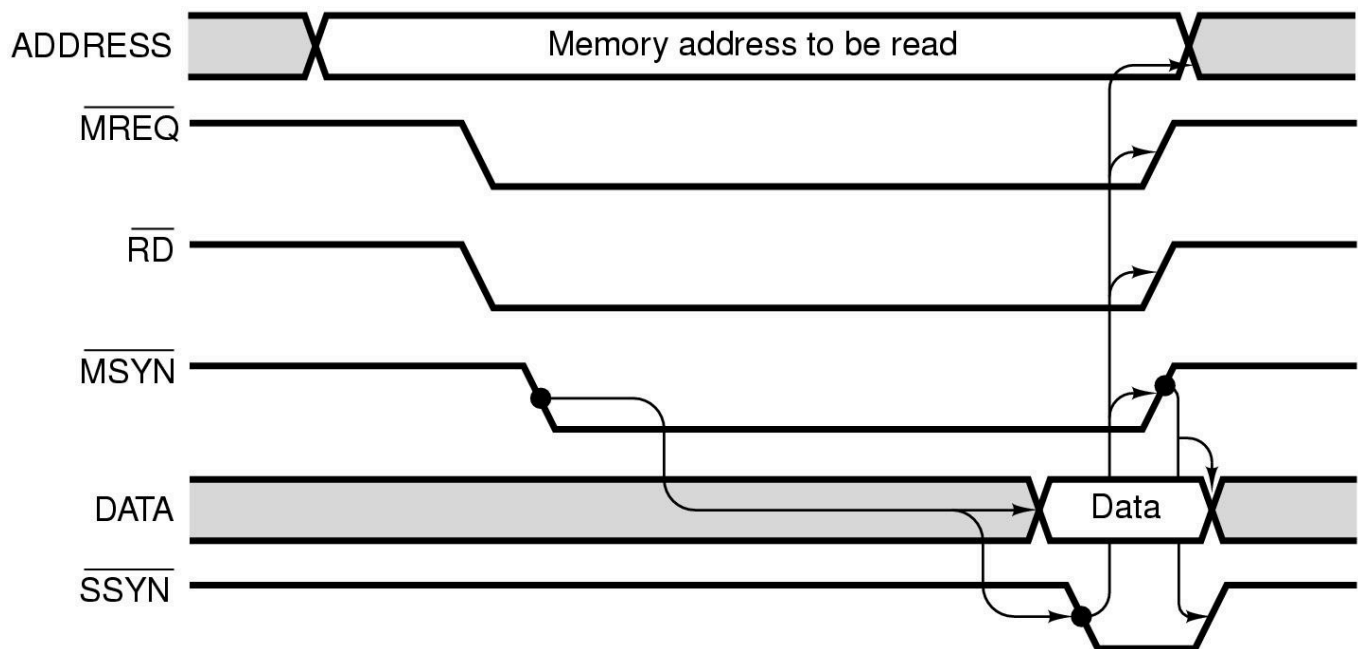
✓ Vantaggi del Bus Sincrono

- ✓ Progetto dei dispositivi più semplice (tempi fissi).
- ✓ Se i tempi sono noti e costanti, **WAIT** può non essere necessario.

✗ Svantaggi del Bus Sincrono

- ✗ La durata dell'operazione deve essere un **multiplo intero del ciclo di clock**.
- ✗ La **velocità del bus è vincolata** al dispositivo più lento (tutti gli altri devono "aspettarlo").

Bus asincroni:



🔧 Cos'è un bus asincrono?

Un **bus asincrono** è un tipo di collegamento tra CPU e memoria (o altri dispositivi) in cui **non esiste un clock condiviso**.

Questo significa che **le operazioni non avvengono a tempo fisso**, ma ogni dispositivo può lavorare con i **propri tempi**.

Questa caratteristica rende il bus asincrono molto **flessibile**, perché ogni operazione ha **la durata necessaria in base alla velocità dei dispositivi coinvolti**.

🤝 Cos'è l'handshake?

Il **protocollo di handshake** (in italiano: "stretta di mano") è un **meccanismo di sincronizzazione** tra due dispositivi (in questo caso, CPU e memoria).

Serve a **coordinare** la trasmissione dei dati senza usare un clock.

Nel **full handshake**, ci sono **quattro passaggi** fondamentali che garantiscono che la comunicazione avvenga correttamente dall'inizio alla fine.

Procedura del ciclo di lettura su bus asincrono

1. La CPU avvia la richiesta

- Attiva i segnali **MREQ** (richiesta memoria) e **RD** (richiesta di lettura).
- Subito dopo attiva anche il segnale **MSYN** (Master Synchronization), per comunicare al dispositivo che vuole leggere dei dati.

2. Il dispositivo risponde

- Quando il dispositivo (ad esempio una memoria) rileva **MSYN attivo**, inizia il suo lavoro.
- Quando ha finito e i dati sono pronti, **attiva il segnale SSYN** (Slave Synchronization), per dire alla CPU che i dati sono disponibili.

3. La CPU legge i dati

- Una volta che SSYN è attivo, la CPU legge i dati dal bus dati.
- Dopo aver letto, **disattiva MREQ, RD e MSYN**, cioè segnala che ha completato l'operazione.

4. Il dispositivo completa il ciclo

- Vedendo che **MSYN è stato disattivato**, anche il dispositivo **disattiva SSYN**.
- A questo punto il **ciclo di lettura è concluso** e il bus è pronto per un'altra comunicazione

Vantaggi del Bus Asincrono

-  **Flessibilità totale**: ogni operazione dura esattamente quanto serve (nessun ciclo fisso).
-  Ottimale per sistemi eterogenei con dispositivi a velocità molto diverse.

Svantaggi del bus asincrono

- Ogni comunicazione richiede **sempre 4 fasi**, anche se velocissima → **overhead maggiore**
- Ogni dispositivo deve implementare la **logica di handshake**, quindi il design hardware è **più complesso** rispetto a un bus sincrono.

Come funziona il daisy chaining?

 **Cos'è l'arbitraggio del bus?**

L'arbitraggio è la **funzione che decide chi può usare il bus** quando **più dispositivi (chiamati "master")** vogliono accedervi **contemporaneamente**.

Serve per evitare conflitti e garantire che il bus venga **usato in modo ordinato**, uno alla volta.

⚙️ Tipi di arbitraggio

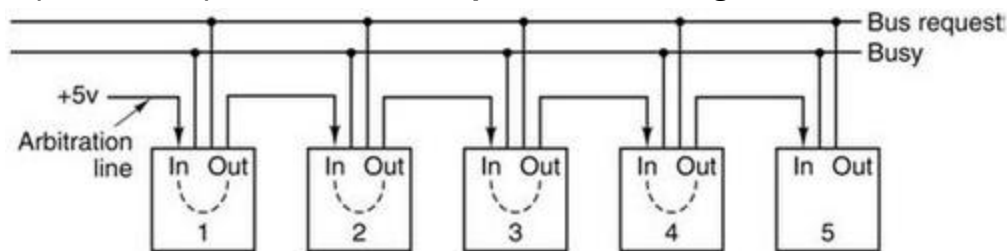
Esistono due principali tipi di arbitraggio:

1. Arbitraggio Centralizzato

- C'è un **arbitro centrale** (spesso nella CPU) che **riceve tutte le richieste** e decide **a chi dare il controllo del bus**.
- È semplice da gestire, ma se l'arbitro si rompe, tutto il sistema può bloccarsi.

2. Arbitraggio Decentralizzato

- **Non c'è un arbitro centrale**.
- I dispositivi **si autogestiscono** secondo una logica di priorità **prestabilita**.
- È più robusto perché **non ha un punto critico di guasto**.



✅ Vantaggi

- **Nessun arbitro centrale** → meno rischi di guasti critici
- **Più flessibile e scalabile**
- Decisioni rapide: i dispositivi **competono direttamente**
- Meno cablaggio per sistemi complessi

❌ Svantaggi

- **Più complesso da implementare** (ogni dispositivo deve sapere confrontare le priorità)
- Richiede **logica di arbitraggio più sofisticata**
- Priorità comunque **fisse** (anche se più eque rispetto al daisy chaining)

🌸 Daisy Chaining (centralizzato)

Il **daisy chaining** è un esempio di **arbitraggio centralizzato**.

Si chiama così perché i dispositivi sono **collegati in catena**, uno dopo l'altro, come in una "collana di margherite".

Come funziona?

1. Un **dispositivo chiede l'uso del bus** al controllore centrale.
2. L'arbitro invia un segnale chiamato **Grant** al **primo dispositivo della catena**.
3. Quel dispositivo:
 - Se **ha fatto richiesta, prende il controllo del bus e non passa Grant avanti**.
 - Se **non ha fatto richiesta, propaga Grant** al dispositivo successivo.
4. In questo modo, **il primo dispositivo che ha fatto richiesta** (cioè il più vicino all'arbitro) **ottiene il bus**.

Vantaggi

- **Semplice da implementare**
- **Economico**: usa poche linee di controllo
- Il controllo è **centralizzato**, quindi più facile da gestire in sistemi piccoli

Svantaggi

- **Priorità fissa**: i dispositivi più vicini all'arbitro hanno **sempre la precedenza**
- **Starvation**: i dispositivi lontani rischiano di **non essere mai serviti**
- Se un dispositivo nella catena si guasta, può **bloccare tutto il sistema**

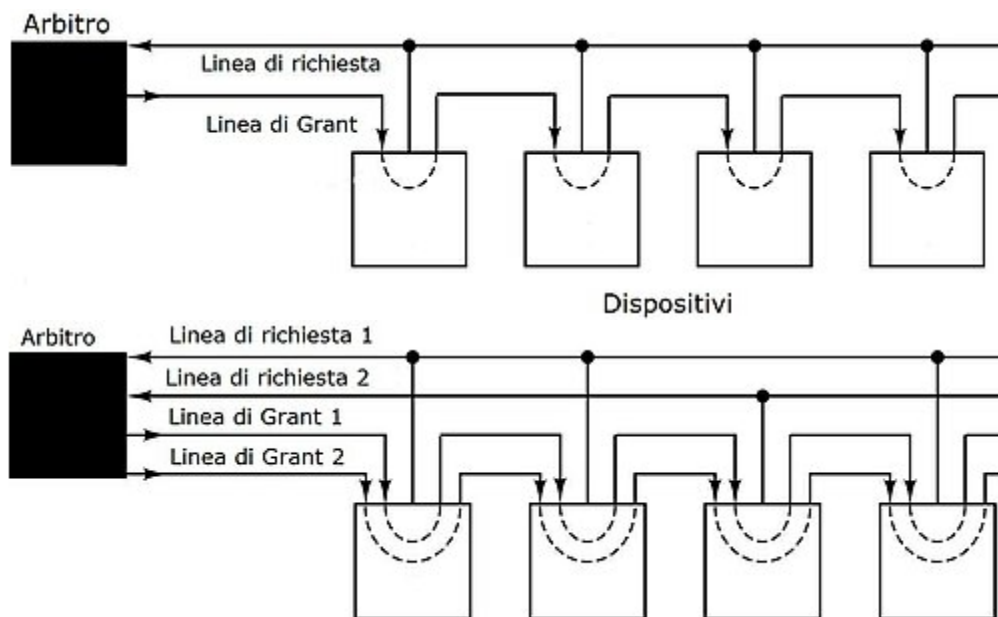
Priorità

In questo sistema, **la priorità dipende dalla posizione fisica nella catena**:

 **più sei vicino all'arbitro, più hai priorità.**

Estensione: livelli di priorità

In sistemi più complessi si possono usare **più linee di richiesta (Request)** e **più linee di concessione (Grant)**, ognuna per un livello di priorità diverso.



Perché il Bus è fonte di ritardo nell'esecuzione dell'istruzione?

Il bus è fonte di ritardo nell'esecuzione dell'istruzione a causa di diversi fattori:

- Propagazione dei segnali elettrici
- Tempo di stabilizzazione dei segnali
- Tempo di accesso alla memoria
- Tempi di attesa

Quali e cosa sono le Trap?

⚙️ Cosa sono le trap

le **trap** sono un tipo particolare di **interruzioni sincrone**, cioè eventi che interrompono il normale flusso del programma e fanno eseguire **una procedura speciale gestita dal sistema operativo**.

Le trap (chiamate anche **eccezioni** o **interrupt sincrni**) sono:

- Interruzioni **generate dal programma stesso**, non da eventi esterni
- Servono per **gestire condizioni eccezionali** oppure per **richiedere un servizio al sistema operativo**
- Sono **sincrone**, cioè **legate all'esecuzione di una specifica istruzione** (ad esempio divisione per zero)

🧠 Chi le gestisce?

Il sistema operativo ha un componente chiamato **gestore delle trap**:

- Quando avviene una trap, il programma **salta a una locazione di memoria predefinita**
- In quella locazione c'è l'**indirizzo del gestore**, cioè una routine che si occupa di rispondere all'evento

Tipi di trap / eccezioni sincrone

Ci sono **tre categorie principali** di trap:

1. **Errore**

- Causato da problemi interni al programma
- Esempi: **overflow**, **divisione per zero**, **accesso a memoria non valida**

2. **Environment Call (o System Call)**

- Usata dal programma per **chiedere un servizio al sistema operativo**
- Esempi: **leggere un numero**, **stampare a schermo**

3. **Environment Break**

- Generata per motivi di **debugging** o diagnostica
- Serve ad esempio per **inserire un punto di interruzione** (breakpoint) nel programma

Cosa fa la CPU quando c'è una Trap?

quando si verifica una **trap**, la **CPU interrompe temporaneamente l'esecuzione del programma** per gestire un evento eccezionale.

Ecco i **passaggi principali** che la CPU compie:

1. Salvataggio del Program Counter (PC)

La CPU salva l'indirizzo dell'istruzione corrente, cioè il **Program Counter**, così da poter riprendere l'esecuzione del programma dopo che la trap è stata gestita.

2. Passaggio all'Exception Handler

Il controllo passa a una **routine speciale** chiamata **Exception Handler**, il **gestore delle eccezioni**, che si trova in una **locazione di memoria fissa**.

3. Modalità protetta (kernel mode)

Il gestore dell'eccezione viene eseguito in **modalità protetta** (chiamata anche **kernel mode**), perché ha accesso diretto alle **risorse del sistema operativo**, ed è fondamentale che:

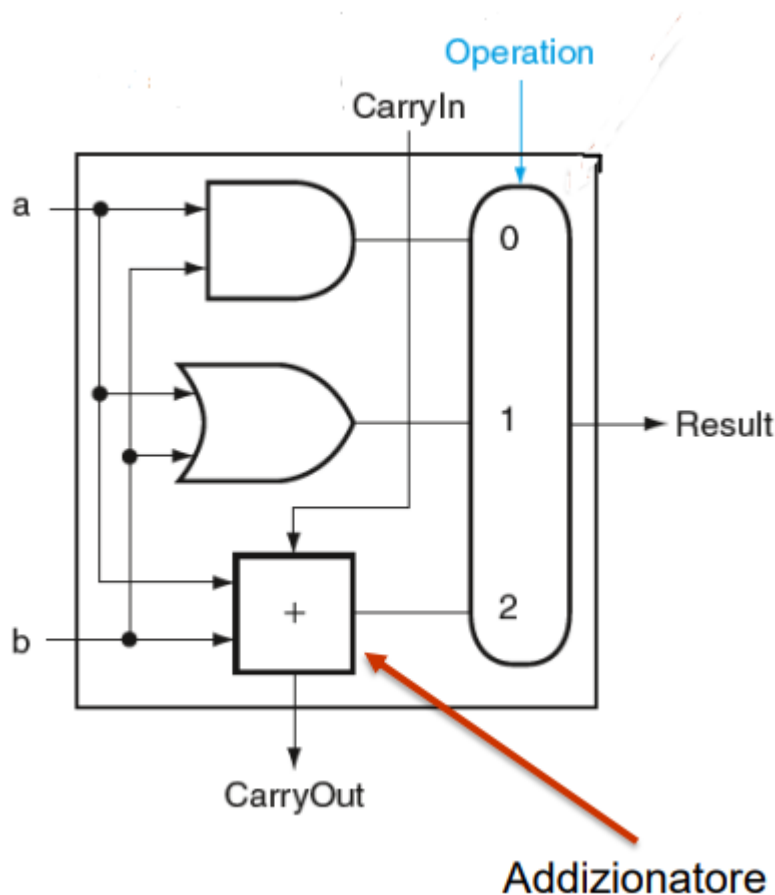
- non alteri **lo stato del programma utente**
- mantenga intatti tutti i **registri della CPU**
- gestisca le eccezioni **una alla volta**

🔧 4. Azione correttiva

A seconda del tipo di trap, l'Exception Handler può:

- Terminare il programma (in caso di errore grave, come divisione per zero)
- Fornire un servizio richiesto (es. stampa a schermo)
- Saltare a una procedura di gestione dell'errore
- Riprendere l'esecuzione dal punto salvato

Cosa c'è dentro una ALU?



⚙️ Componenti interni di una ALU a 1 bit

- **Ingressi dati:**
 - Due ingressi A e B, ciascuno di 1 bit, che rappresentano i due operandi su cui si vuole operare.
- **Ingresso di controllo (Operation):**
 - Un ingresso a 2 bit che seleziona quale operazione deve essere eseguita (ad esempio AND, OR, somma).
- **Ingresso CarryIn:**

- Un bit che rappresenta il riporto in ingresso (utile per operazioni di somma multi-bit, per concatenare più ALU a 1 bit).
- **Uscita risultato:**
 - Un bit che contiene il risultato dell'operazione scelta.
- **Multiplexer:**
 - Un circuito che, in base al segnale di controllo, seleziona quale operazione (AND, OR, somma, ecc.) deve essere trasmessa in uscita.
- **Porta AND:**
 - Realizza l'operazione logica AND tra A e B.
- **Porta OR:**
 - Realizza l'operazione logica OR tra A e B.
- **Circuito Half-Adder:**
 - Realizza la somma binaria di A e B con il CarryIn, generando somma e riporto (CarryOut).

risc-V

A cosa serve lo stack e perché è stato inventato da chi ha progettato Risc-V?

Lo stack è una struttura dati dinamica, un'area di memoria in cui è possibile salvare i registri per evitarne la perdita del valore. L'accesso è di tipo LIFO, la cima dello stack è identificata dallo Stack Pointer (SP). In RISC-V si usa la convenzione:

 Perché serve lo stack in RISC-V (e in generale)

✓ 1. Supporto a funzioni e ricorsione

Quando una funzione viene chiamata, bisogna salvare:

- l'indirizzo di ritorno (`ra`)
- i registri che verranno temporaneamente usati
- eventuali argomenti e variabili locali

Tutto questo viene **spinto (push)** nello stack. Al termine della funzione, i dati vengono **estratti (pop)** per ripristinare lo stato precedente.

✓ 2. Gestione ordinata del contesto

Immagina una funzione `A()` che chiama `B()` , che chiama `C()` :

- Ogni funzione ha **propri registri temporanei, variabili locali, indirizzo di ritorno.**

- Lo stack permette di **ripristinare l'esatto contesto** della funzione chiamante quando la funzione chiamata termina.

✅ 3. Modularità e riusabilità

Grazie allo stack:

- Le funzioni sono **riutilizzabili**: ognuna può usare gli stessi registri senza preoccuparsi di conflitti.
- Il compilatore può generare codice efficiente e portabile.

Come si usa lo stack in RISC-V

- RISC-V usa il **registro sp (stack pointer, x2)** per puntare alla cima dello stack.
- Lo stack **cresce verso indirizzi più bassi**.

Domanda	Risposta
Cos'è lo stack?	Una struttura LIFO per gestire dati temporanei e contesto funzione
Perché esiste in RISC-V?	Per supportare funzioni, ricorsione e modularità

Perché ho sia Frame pointer che Stack pointer, e non solo sp?

Registro	Nome	Funzione principale
sp (x2)	Stack Pointer	Punta alla cima dello stack , cambia dinamicamente
fp (x8/s0)	Frame Pointer	Punta all' inizio del frame attuale della funzione

Perché non usare solo sp ?

Perché lo **sp cambia continuamente** durante l'esecuzione della funzione:

- Quando si salvano registri: **sp** scende
- Quando si alloca spazio per variabili locali: **sp** scende
- Quando si chiama una funzione annidata: **sp** scende ancora
 - 👉 Quindi **gli offset da sp diventano instabili**: difficile accedere a una variabile locale con un offset fisso.

✓ A cosa serve il Frame Pointer

Il **Frame Pointer** (`fp` o `sp`) resta **fisso** per tutta la durata della funzione. Questo consente:

1. 🔍 **Accesso semplice e stabile** alle variabili locali e ai parametri:
 - es. `lw t0, -8(fp)` → sempre la stessa variabile
2. 🐛 **Debug semplificato**:
 - Un debugger può facilmente camminare indietro nella *call stack* grazie a `fp`
3. 🔄 **Funzioni ricorsive o annidate**:
 - Ogni chiamata può avere il proprio `fp`, rendendo la gestione dello stack chiara

Motivo per cui esistono entrambi	Spiegazione
<code>sp</code> cambia frequentemente	Non è stabile per accesso a variabili
<code>fp</code> è stabile	Accesso a parametri e locali con offset fissi
Aiuta il debug	Permette a debugger e profiler di camminare nello stack
Non è sempre obbligatorio	Il compilatore può decidere di non usarlo per ottimizzare (caso funzioni semplici)

Spiegare anche FP, GP, PC

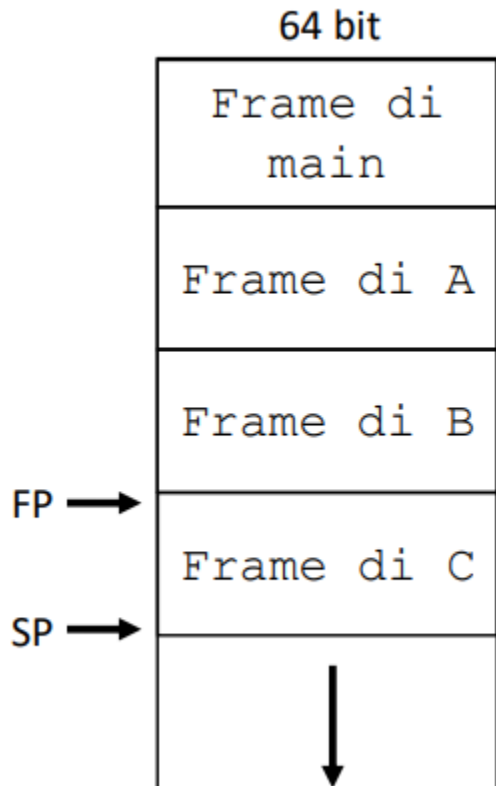
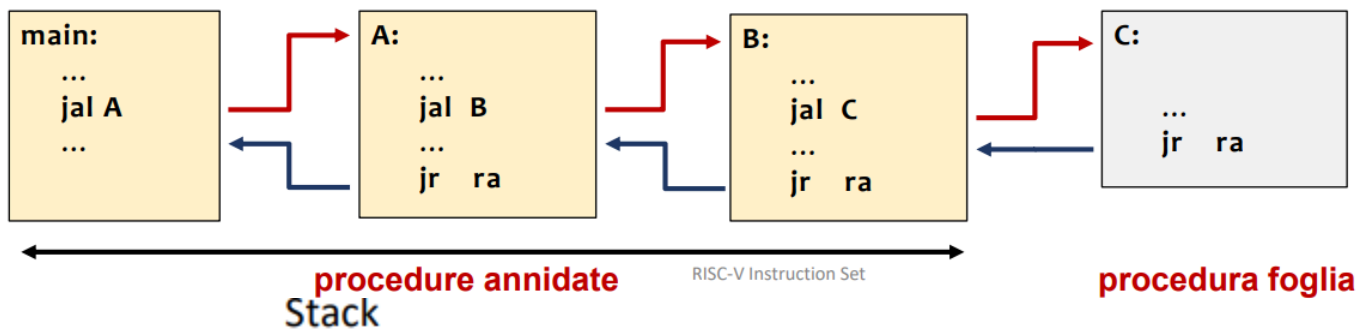
- Frame Pointer (FP): serve ad identificare la base di un frame
- Global Pointer (GP): serve ad identificare l'indirizzo della parte della memoria dedicata ai dati statici (globali)
- Program Counter (PC): serve ad identificare l'indirizzo della prossima istruzione da eseguire

Spiegare come funziona il record di attivazione

Ad ogni esecuzione di procedura corrisponde un blocco di memoria all'interno dello stack che viene chiamato record di attivazione o frame.

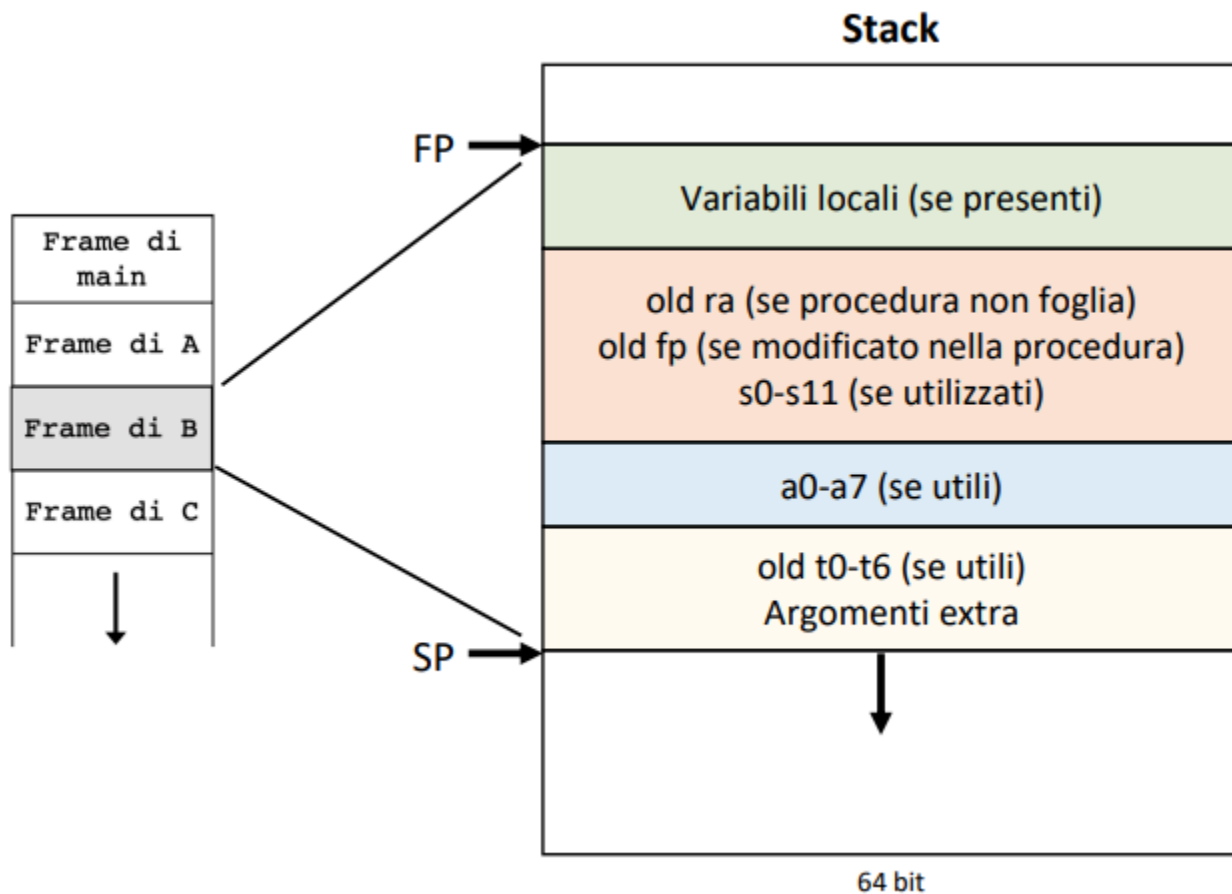
Quando viene richiamata una funzione F, viene aggiunto un record di attivazione allo stack.

Quando la funzione F termina, il corrispondente record di attivazione viene cancellato



Il frame in cima allo stack rappresenta il frame della procedura in esecuzione. I confini del frame sono identificati da:

- Base: il Frame Pointer (FP)
- Cima: lo Stack Pointer (SP)



Se utilizzato, il Frame Pointer viene inizializzato al valore di sp all'inizio della chiamata. Fp consente di avere un riferimento alle variabili locali che non muta con l'esecuzione della procedura.

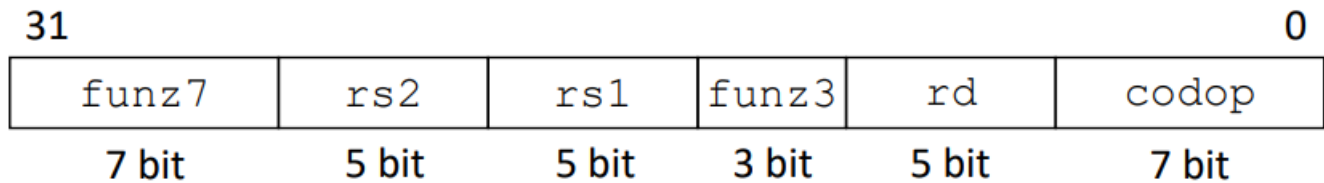
Se lo stack non contiene variabili locali alla procedura, il compilatore risparmia tempo di esecuzione evitando di impostare e ripristinare il frame.

Perché esistono tanti tipi di istruzioni su Risc-V e a cosa servono?

RISC-V definisce diversi formati di istruzioni che consentono di codificare in binario ogni istruzione assembler, ogni istruzione RISC-V richiede esattamente 32 bit. I formati delle istruzioni RISC-V sono R, I, S, SB, U:

Tipo	Nome	A cosa serve	Esempio
R-type	Aritmetica tra registri	Somma, AND, XOR tra registri	<code>add x1, x2, x3</code>
I-type	Immediate	Operazioni con costanti, load, syscall	<code>addi x1, x2, 10</code>
S-type	Store	Scrive dati da registri alla memoria	<code>sw x3, 0(x2)</code>
B-type	Branch	Salti condizionati	<code>beq x1, x2, label</code>
U-type	Upper Immediate	Caricamenti di costanti grandi	<code>lui x1, 0x10000</code>
J-type	Jump	Salti non condizionati (es. funzione)	<code>jal x1, label</code>

Formato di tipo R (registro) Permette di codificare le istruzioni tra due registri sorgente.



codop: codice operativo dell'istruzione

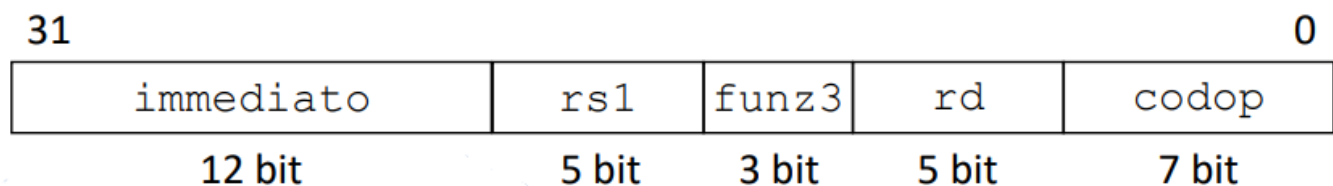
rd: registro di destinazione

rs1: registro che contiene il primo operando sorgente

rs2: registro che contiene il secondo operando sorgente

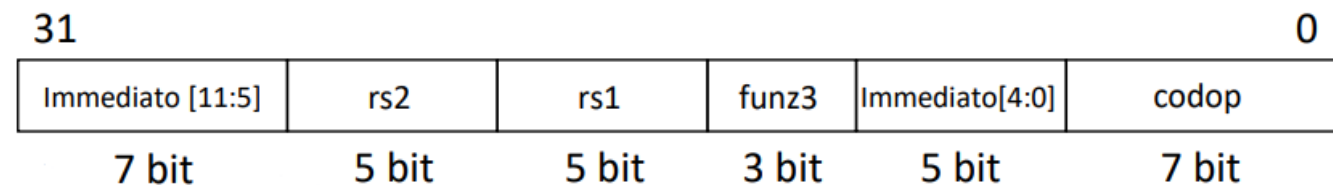
funz3, funz7: codici operandi aggiuntivi

Formato di tipo I (immediato) Permette di codificare le istruzioni che richiedono il caricamento dalla memoria o una costante



Sono presenti 12 bit perché con 5 bit l'intervallo di rappresentazione per costanti e (soprattutto) offset sarebbe stato troppo ridotto. Il campo immediato è rappresentato in complemento a due (valori possibili da -2048 a +2047).

Formato di tipo S Permette di codificare le istruzioni che richiedono il salvataggio in memoria o una costante.



Il campo immediato viene diviso in due parti per mantenere i due campi rs1 e rs2 nella stessa posizione rispetto alle istruzioni di tipo R. Il campo immediato è rappresentato in complemento a due (valori possibili da -2048 a +2047).

Formato di tipo SB:

Permette di codificare le istruzioni di salto condizionato. Il formato può rappresentare indirizzi di salto da -4096 a 4094, in multipli di due.

Formato di tipo U:

Permette di codificare le istruzioni di salto condizionato con indirizzo di salto più grandi, un intervallo pari a 232 .

Domanda	Risposta
Perché esistono più tipi di istruzioni?	Per gestire operazioni diverse con efficienza e chiarezza
Perché hanno formati diversi?	Per ottimizzare la decodifica e ridurre la complessità hardware
Cosa guadagno con questo?	Codice più compatto, hardware più semplice, architettura scalabile/modulare

Perché evitare di usare la memoria in Risc-V? Quindi come funziona la memoria, dove si trovano memoria e dove registri?

È consigliabile limitare le istruzioni di accesso alla memoria il più possibile, dato che l'accesso alla memoria è più lento rispetto a quello dei registri.

La memoria è un insieme di celle della stessa dimensione, ognuna di esse è identificata da un indirizzo, in RISC-V la memoria è organizzata in parole da 64 bit, in essa possiamo trovare:

- Stack
- Dati allocati dinamicamente (heap)
- Dati statici e variabili globali
- Codice del programma

Lo stack e i dati dinamici (heap) crescono in direzioni differenti, in modo da ottimizzarne la gestione.

I registri si trovano all'interno della CPU.

Differenza tra jump e jump and link?

Jump: salta da un'istruzione a quella da eseguire, prosegue dall'istruzione immediatamente successiva a quella a cui si è "saltati".

Jump-and-link: salta da un'istruzione a quella da eseguire, appena termina l'esecuzione dell'istruzione a cui si è saltati, l'istruzione seguente da eseguire è quella subito dopo l'istruzione di jump-and-link.

Ci sono istruzioni più veloci di altre?

Sì, ci sono istruzioni che sono più veloci di altre, questo dipende dal numero di cicli di clock per istruzione (CPI), ovvero il numero medio di cicli di clock per istruzione di un programma o di un frammento di programma.

Cosa sono le convenzioni di chiamata?

le **convenzioni di chiamata** in RISC-V sono un insieme di regole che definiscono **come i parametri delle funzioni vengono passati, come viene restituito il valore di ritorno, e come gestire il salvataggio dei registri** durante una chiamata di funzione.

Come funzionano le convenzioni di chiamata in RISC-V?

1. Passaggio dei parametri

- I primi 8 parametri di una funzione vengono passati nei registri **a0 - a7** (che corrispondono ai registri x10 - x17).
- Se ci sono più di 8 parametri, quelli in eccesso vengono passati **sullo stack**.

2. **Valore di ritorno

- Il valore restituito dalla funzione viene messo nel registro **a0** (x10).
- Se la funzione restituisce valori più complessi (ad esempio strutture), si usano più registri o lo stack, a seconda dei casi.

3. Salvataggio dei registri

- I registri **temporanei** (t0 - t6) non devono essere preservati dal chiamato, quindi il chiamante deve salvarli se li usa ancora dopo la chiamata.
- I registri **saved** (s0 - s11) devono essere salvati e restaurati dal chiamato se li modifica, per preservare lo stato.

4. Stack frame e indirizzo di ritorno

- L'indirizzo di ritorno viene salvato nel registro **ra** (x1).
- La funzione, se ha bisogno di usare lo stack, crea un **frame sullo stack** dove salva i registri necessari, inclusi **ra** e i registri **s*** che modifica.

Come viene aggiornato il Program Counter?

Il **Program Counter** (PC) è un registro che contiene l'indirizzo dell'istruzione corrente che la CPU sta eseguendo.

Aggiornamento normale (sequenziale)

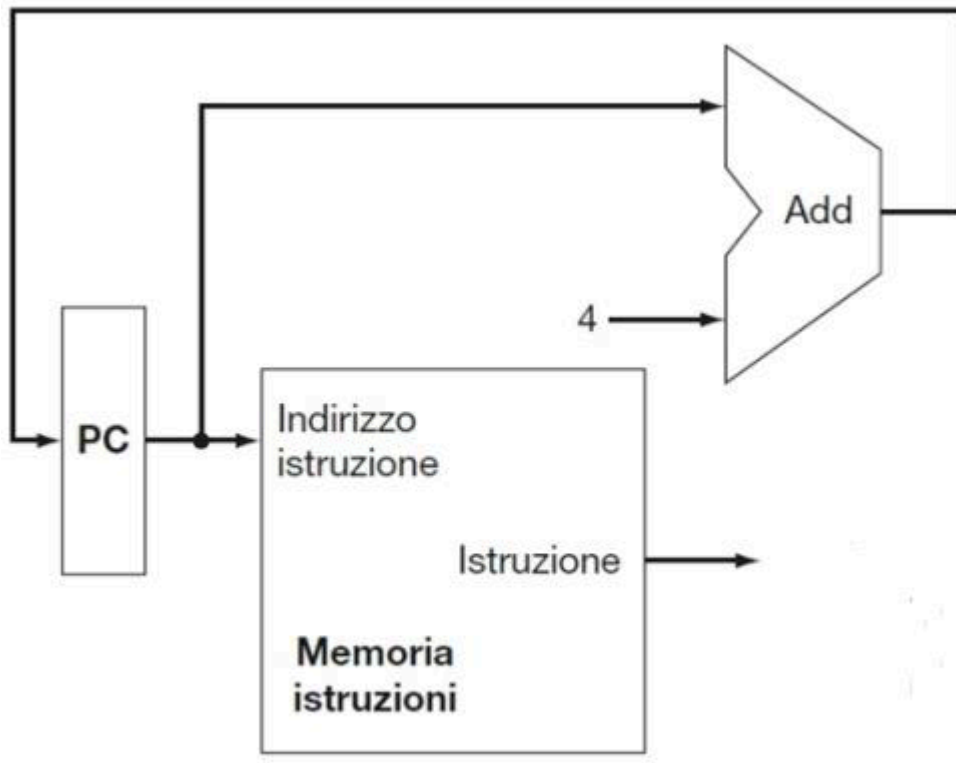
- In condizioni normali, dopo aver eseguito un'istruzione, il PC viene aggiornato **automaticamente** per puntare all'istruzione successiva.
- Questo aggiornamento avviene aggiungendo **4 byte** (o 32 bit) all'indirizzo attuale, perché in architettura **RISC-V** ogni istruzione è lunga esattamente 4 byte.
- Tecnicamente, un **sommatore** interno calcola:

$$PC_{next} = PC_{current} + 4$$

Aggiornamento in caso di salto (branch/jump)

- Quando l'istruzione è un salto o un branch, il PC non viene incrementato di 4, ma viene caricato con un **nuovo indirizzo** specificato dall'istruzione stessa.
- Questo nuovo indirizzo può essere calcolato in vari modi, ad esempio:

- Somma tra il PC attuale e un offset (per i branch relativi)
- Un indirizzo assoluto specificato direttamente (per i jump)



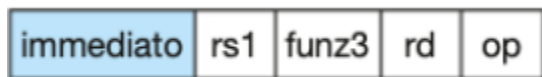
Quali sono i modi di indirizzamento?

I modi di indirizzamento sono convenzioni utilizzate nella programmazione in linguaggio assembly che specificano come certi valori vengono letti in un determinato segmento di istruzioni per il montaggio.

Abbiamo quattro modi di indirizzamento principali:

- Indirizzamento immediato
- Indirizzamento tramite registro
- Indirizzamento tramite base e spiazzamento
- Indirizzamento relativo al program counter

Indirizzamento immediato



L'indirizzamento immediato si ha quando all'interno di un'istruzione si opera con una costante numerica

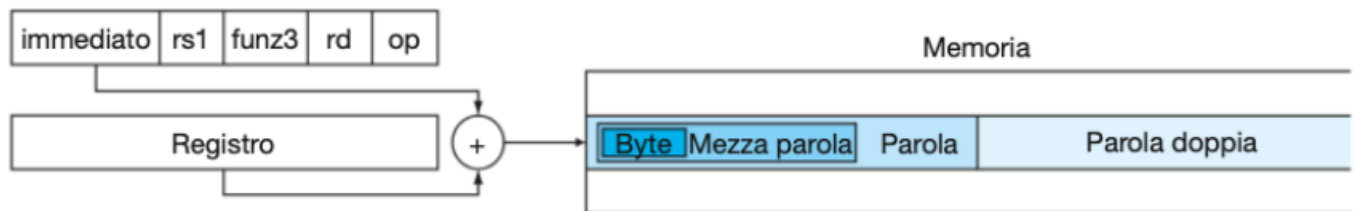
Indirizzamento tramite registro



L'indirizzamento a registro si ha quando gli operandi dell'istruzione sono entrambi registri della CPU.

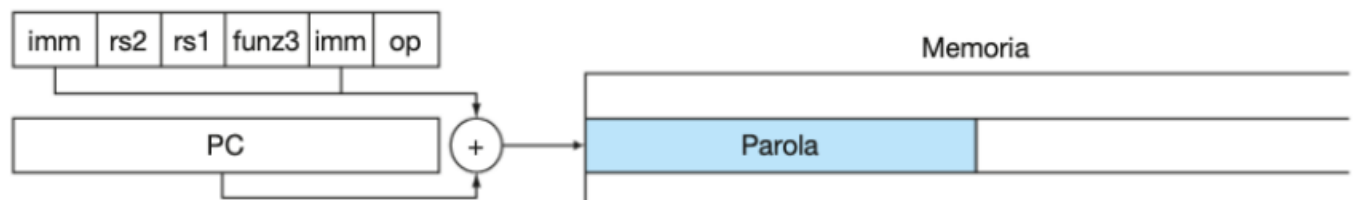
Indirizzamento tramite base e spiazzamento

L'indirizzamento tramite base e spiazzamento è utilizzato per recuperare un dato dalla memoria che si trova ad un offset (campo immediato) rispetto ad un indirizzo contenuto nel registro utilizzato nell'istruzione.

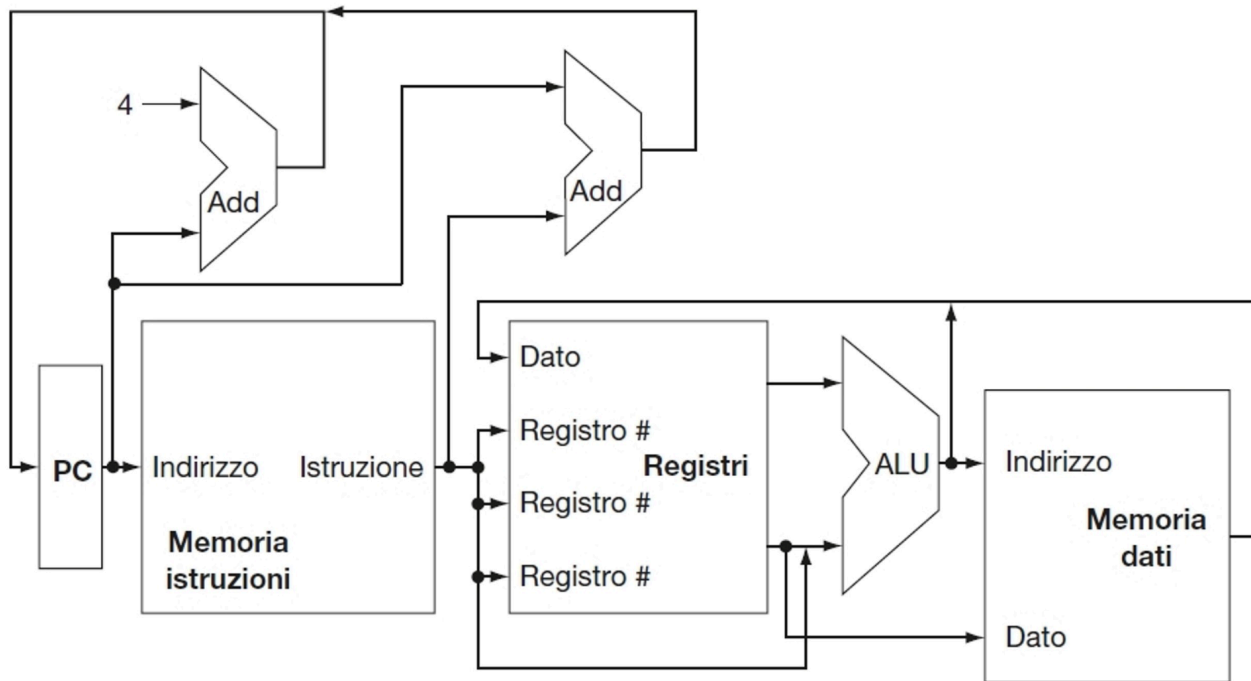


Indirizzamento relativo al program counter

L'indirizzamento relativo al program counter viene utilizzato per ottenere il prossimo valore di PC in caso di salto, eseguendo il calcolo $PC = PC + \text{immediato}$.



data path RISC-V (SEMPLIFICATO)



1. PC e Memoria Istruzioni

- Il **Program Counter (PC)** tiene l'indirizzo della **prossima istruzione** da eseguire.
- L'indirizzo viene inviato alla **memoria istruzioni**, che restituisce l'**istruzione** da eseguire.
- Dopo la lettura, il **PC viene aggiornato** con:
 - $PC + 4$ per passare all'istruzione successiva (in caso di esecuzione sequenziale).
 - Oppure con un **nuovo indirizzo** (se c'è un salto/branch).

2. Register File (Registri)

- L'istruzione contiene 3 campi importanti:
 - Due numeri di **registri sorgente**.
 - Un numero di **registro destinazione**.
- Questi vengono usati per leggere due valori dal **registro file** (i due sorgenti) e poi scrivere eventualmente un valore nel registro destinazione.
- Il valore da scrivere arriva dall'output dell'**ALU** o dalla **memoria dati**, a seconda del tipo di istruzione.

3. ALU (Arithmetic Logic Unit)

- Esegue operazioni aritmetiche/logiche tra:
 - Due registri.
 - Oppure un registro e un valore immediato.
- Il risultato può essere:
 - Usato per scrivere in un registro.

- Usato come **indirizzo** per la **memoria dati** (load/store).



4. Memoria Dati

- Se l'istruzione è un `load`, l'**indirizzo** calcolato dalla ALU viene usato per leggere un dato dalla **memoria dati**.
- Se è uno `store`, viene scritto un valore in memoria (usando l'indirizzo ALU e il valore da un registro).

+ 5. Adders (somma)

- Ci sono **due adders** nello schema:
 - Uno somma `PC + 4` per passare all'istruzione successiva.
 - L'altro probabilmente serve per calcolare un **indirizzo di branch** (`PC + offset`), usato nei salti condizionati o in `jal`, `beq`, ecc.



Componenti sequenziali vs combinatori

Componente	Tipo	Note
PC	Sequenziale	Tiene lo stato (indirizzo attuale)
Memoria istruzioni	Sequenziale	Ha contenuto statico, ma viene letta sequenzialmente
Registri	Sequenziale	Scrittura sincronizzata (flip-flop)
Memoria dati	Sequenziale	Accesso sincrono (scrittura/lettura)
ALU	Combinatorio	Nessun stato interno
Adders (+)	Combinatori	Sommano due input



FASI LOGICHE DEL CICLO DI ISTRUZIONE

◆ 1. FETCH

Obiettivo: caricare l'istruzione dalla memoria.

- **Componenti coinvolti:**
 - **PC (Program Counter):** contiene l'indirizzo dell'istruzione da eseguire.
 - **Memoria delle Istruzioni:** riceve l'indirizzo dal PC → restituisce l'istruzione.
 - **Instruction Register** (non presente nello schema, ma concettualmente esiste): memorizza temporaneamente l'istruzione letta per le fasi successive.
- **Nota:**

- La memoria delle istruzioni è **read-only**, quindi ha **solo ingresso di indirizzo e uscita dati** (nessuna scrittura).
- Il **PC viene aggiornato** con $PC + 4$ (con l'ALU dedicata all'incremento), a meno che non ci sia un salto.

◆ 2. DECODE

Obiettivo: decodificare l'istruzione e preparare gli operandi.

- **Componenti coinvolti:**
 - **Control Unit:** interpreta l'istruzione e genera segnali di controllo per gli altri componenti (ALU control, MemRead, MemWrite, RegWrite, ALUSrc, ecc.).
 - **Register File:** usa i numeri dei registri contenuti nell'istruzione per leggere i **registri sorgente**.
- **Nota:**
 - Il **decoder** seleziona i registri da leggere.
 - Gli **operandi** sono pronti per la fase di esecuzione.

◆ 3. EXECUTE

Obiettivo: eseguire l'operazione logica/aritmetica o calcolare un indirizzo.

- **Componenti coinvolti:**
 - **ALU principale:** esegue operazioni come somma, AND, confronto, ecc.
 - **ALU secondaria (Add):** calcola l'indirizzo in caso di salti ($PC + offset$) → giustifica la presenza della seconda ALU nello schema.
- **Comportamento in base al tipo di istruzione:**
 - **Istruzioni aritmetico-logiche (es. add, and, slt)** → ALU calcola e il risultato va scritto in un **registro**.
 - **Istruzioni di memoria (lw, sw)** → ALU calcola l'**indirizzo**:
 - **lw**: carica un dato dalla memoria in un registro.
 - **sw**: salva un dato da un registro nella memoria.
 - **Salti condizionati (beq, bne, ecc.):**
 - ALU principale confronta i due operandi.
 - Se la condizione è vera, l'**ALU additiva** calcola il nuovo valore del PC ($PC + offset$).



Riflessioni importanti

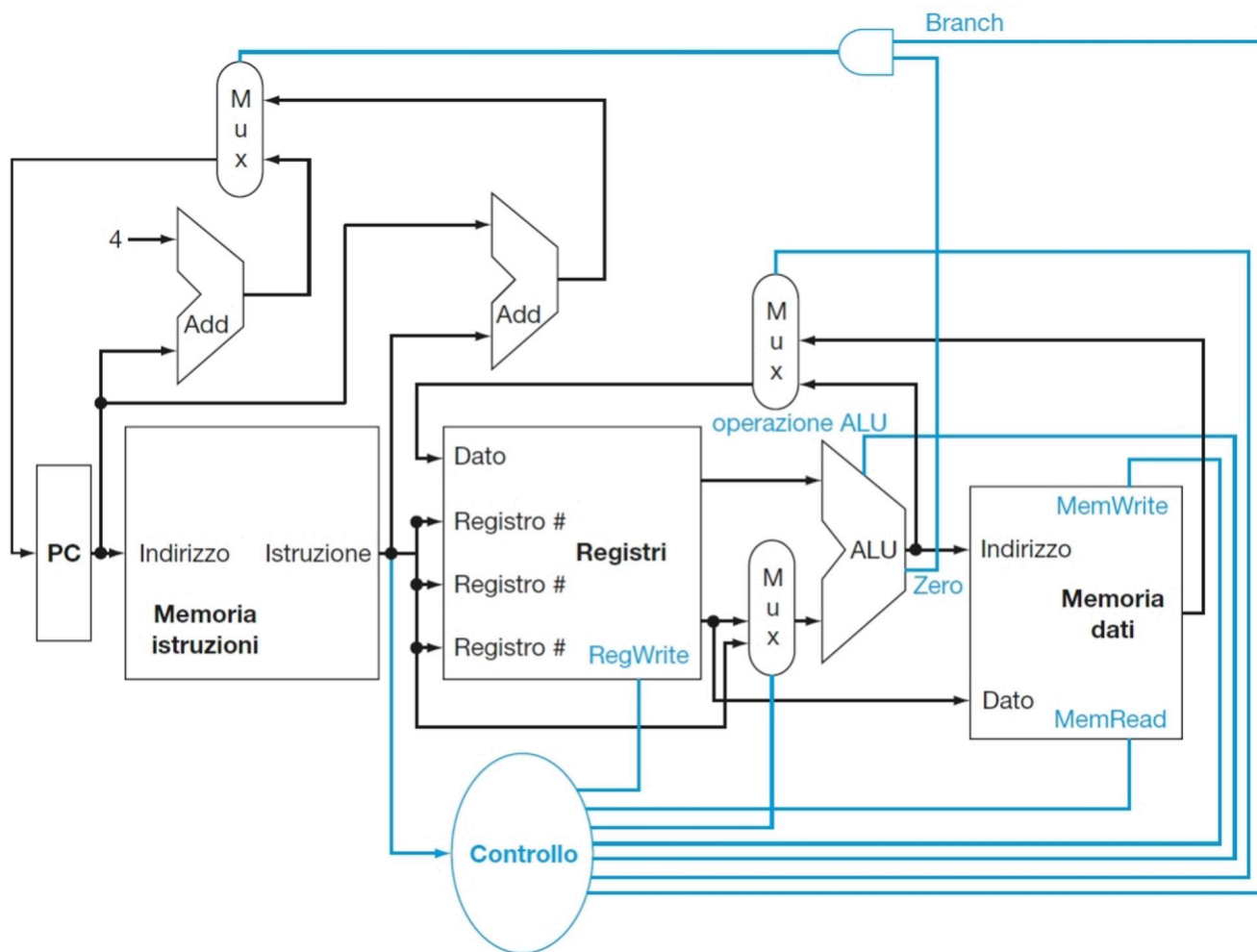
- Le **fasi non sono scandite da clock**: sono solo una **divisione concettuale**, non si tratta di una pipeline.

- L'**effettiva esecuzione** (comprese letture/scritture) avviene tutta in un singolo ciclo, in questo modello semplificato.
- Le **ALU** sono **combinatorie**: il risultato dipende solo dagli input, non c'è memoria.
- I componenti come il **PC**, i **registri** e le **memorie** sono **sequenziali**: aggiornano i valori solo in base al clock e ai segnali di controllo.

✚ Riassunto dei componenti per fase

Fase	Componenti principali	Tipo	Funzione
FETCH	PC, Memoria Istruzioni, ALU (+4)	Sequenziali + combinatori	Recupero dell'istruzione
DECODE	Control Unit, Register File	Sequenziali + logica	Lettura registri, generazione segnali
EXECUTE	ALU, ALU (add), Memoria Dati	Combinatori + sequenziali	Calcolo, accesso alla memoria, aggiornamento registri/PC

L'UNITA' DI CONTROLLO



🔧 UNITÀ DI CONTROLLO: COSA FA?

▶ Funzione

L'unità di controllo (Control Unit) è il "cervello" della CPU:

- Decodifica le istruzioni (in base all'opcode).
- Genera i **segnali di controllo** per:
 - **ALU**
 - **Register File**
 - **Memoria dati**
 - **Scrittura PC**
 - **MUX**
- Decide cosa fare in base al **tipo di istruzione** (R, I, S, B, J, U).

⚙️ Tipi di segnali di controllo generati

Segnale	Funzione
RegWrite	Scrivere in un registro
MemRead	Leggere dalla memoria dati
MemWrite	Scrivere nella memoria dati
MemToReg	Seleziona tra uscita ALU o memoria per scrittura reg.
ALUSrc	Seleziona tra secondo registro o immediato
ALUOp	Imposta il tipo di operazione per la ALU
Branch	Abilita salto condizionato
PCSrc	MUX: decide se usare PC+4 o PC+offset

TEMPORIZZAZIONE EDGE-TRIGGERED

Cosa significa edge-triggered

- Il sistema è **sincrono**: tutto avviene **solo al fronte di salita** del clock.
- I **flip-flop (D)** usati nei registri, PC, memorie reagiscono solo a quel momento preciso.
- Questo meccanismo evita **problemi di trasparenza** (tipici dei latch **level-sensitive**).

Due casi di utilizzo

1. `add x1, x2, x3`

→ Lettura dei registri x2 e x3, somma, e scrittura del risultato in x1.

→ **FLIP-FLOP 1 (lettura)** → **logica combinatoria (ALU)** → **FLIP-FLOP 2 (scrittura)**

2. `add x1, x1, x2`

→ Si legge e scrive lo **stesso** registro (x1), ma non è un problema grazie all'**edge-triggering**:

la **lettura avviene prima**, la **scrittura dopo**, nello stesso ciclo.

IL RUOLO DELL'ADDER PER IL PC

Perché serve un adder?

- Le istruzioni in RISC-V sono larghe **4 byte (32 bit)**.
- Per passare da un'istruzione all'altra bisogna fare:

$$PC = PC + 4$$

Com'è fatto il circuito

- Il valore del **PC** va:
 - Alla **memoria istruzioni** (per leggere l'istruzione corrente).
 - All'**adder**, che **somma 4**.
- L'output dell'adder torna al PC (via MUX, se ci sono salti).

COME AVVIENE IL FETCH AD OGNI CICLO?

1. **PC** invia un indirizzo alla **memoria istruzioni**.
2. L'**istruzione** viene letta e inviata alla fase di decode.
3. Il **PC viene duplicato** → un ramo va all'adder che **somma 4**
4. L'output dell'adder torna pronto per essere scritto nel **PC al prossimo fronte di clock**.
5. Al **fronte di salita** successivo, il nuovo indirizzo è scritto nel PC, e si carica la **nuova istruzione**.

VANTAGGIO DELL'EDGE-TRIGGERING

Permette:

- **Lettura e scrittura nello stesso ciclo di clock**.
- Nessun conflitto o trasparenza nei registri o nel PC.
- Predicibilità e stabilità nei circuiti digitali sincroni.

DATAPATH ISTRUZIONI TIPO R

COMPONENTI COINVOLTI

Componente	Funzione specifica
Program Counter (PC)	Contiene l'indirizzo dell'istruzione corrente
Instruction Memory	Legge l'istruzione all'indirizzo PC
Instruction Register	Memorizza l'istruzione letta
Register File	Legge x2 e x3, scrive il risultato in x1
ALU	Esegue l'operazione indicata (es: add)
Control Unit	Decodifica e genera i segnali di controllo
Adder per PC+4	Calcola il prossimo indirizzo (PC + 4)
Segnali di controllo	RegWrite , ALUOp (4 bit)

FLUSSO OPERATIVO (passo per passo)

1. FETCH

- Il **PC** fornisce l'indirizzo.
- La **Instruction Memory** legge l'istruzione `add x1, x2, x3`.
- L'**Instruction Register** riceve e mantiene l'istruzione.

2. DECODE

- La **Control Unit** analizza l'istruzione
- - Imposta `RegWrite = 1`
 - Imposta `ALUOp = 0010` (ad esempio per `add`)
- Dal formato R, ricava:
 - `rs1 = x2` (registro sorgente 1)
 - `rs2 = x3` (registro sorgente 2)
 - `rd = x1` (registro destinazione)

3. READ REGISTRI

- Il **Register File** legge `x2` e `x3`.
 - I 5 bit di `rs1` e `rs2` sono usati come selettori (MUX).
- I valori letti vengono inviati alla **ALU**.

4. ESECUZIONE

- La **ALU**, pilotata da `ALUOp`, esegue `x2 + x3`.
- L'**output della ALU** sarà pronto **prima del fronte di salita successivo**.

5. SCRITTURA REGISTRO

- Al **prossimo fronte attivo del clock**:
 - Se `RegWrite = 1` → il dato in uscita dalla ALU viene **scritto in x1**.



SEGNALI DI CONTROLLO

Segnale	Stato	Significato
<code>RegWrite</code>	1	Abilita la scrittura sul registro <code>rd</code>
<code>ALUOp</code>	4 bit	Codifica operazione: <code>0010</code> per <code>add</code> , <code>0110</code> per <code>sub</code>
<code>rs1</code> , <code>rs2</code>	5 bit	Selezione registri sorgente

Segnale	Stato	Significato
rd	5 bit	Selezione registro destinazione

Tutti **devono essere stabili prima del fronte attivo**.

TIMING E STABILITÀ

Evento	Quando avviene
Lettura istruzione da memoria	Dopo il fronte di salita
Generazione segnali di controllo	In parallelo alla lettura
Lettura registri sorgente (x2, x3)	Dopo decode, con MUX stabili
Output ALU pronto	Dopo ritardo combinatorio
Scrittura in x1	Al prossimo fronte di clock

💡 Tutto si basa sul fatto che **il fronte attivo è unico**, e che i **ritardi** di ALU, Control Unit e Register File sono **tutti inferiori al periodo del clock** (ben progettato).

COSA FANNO LE ISTRUZIONI LOAD E STORE

- **LOAD**: legge un dato dalla **memoria dei dati** e lo scrive in un **registro**.
- **STORE**: prende un dato da un **registro** e lo scrive nella **memoria dei dati**.

RUOLO DELL'ALU NELLE LOAD/STORE

- L'**ALU** serve a **calcolare l'indirizzo di memoria** da usare.
- Fa una **somma**: $\text{indirizzo} = \text{valore del registro base} + \text{offset (immediato)}$
 $\text{indirizzo} = \text{valore del registro base} + \text{offset (immediato)}$

PARTICOLARITÀ DELL'OFFSET

- L'**offset** (campo immediato) è codificato **in modo diverso** per LOAD e STORE:
 - **LOAD (tipo I)**: i 12 bit dell'immediato sono **contigui** nell'istruzione.
 - **STORE (tipo S)**: i 12 bit sono **divisi** tra due campi (per mantenere la posizione fissa dei registri).

ESTENSIONE DEL SEGNO

- L'**offset** è a 12 bit → deve essere **esteso a 64 bit** (nell'architettura a 64 bit).

- Si fa l'**estensione del segno**:
 - Se il bit più significativo dei 12 è 0 → si estende con 0.
 - Se è 1 → si estende con 1.
- Il **circuito di estensione del segno** è **combinatorio**, quindi non ha bisogno di memoria o di conoscere le istruzioni precedenti.

CONTROL UNIT

La **Control Unit** controlla:

- **MemRead**: 1 se si fa una LOAD.
- **MemWrite**: 1 se si fa una STORE.
- Il segnale che dice all'ALU di fare una **somma**.

ESEMPIO: `sd x2, 8(x1)`

- Obiettivo: scrivere in memoria il contenuto di `x2` all'indirizzo `x1 + 8`.
- Passaggi:
 1. Leggo `x1` e `x2` dal **register file**.
 2. Faccio la somma: `ALU(x1, 8) → indirizzo`.
 3. Mando `x2` come **dato da scrivere** in memoria.
 4. La **Control Unit** imposta `MemWrite = 1`.

ESEMPIO: `ld x2, 8(x1)`

- Obiettivo: leggere dalla memoria l'indirizzo `x1 + 8` e salvare il valore in `x2`.
- Passaggi:
 1. Leggo `x1` dal **register file**.
 2. Faccio la somma: `ALU(x1, 8) → indirizzo`.
 3. La memoria restituisce il **dato**.
 4. Lo salvo in `x2`.
 5. La **Control Unit** imposta `MemRead = 1`.

BEQ: ISTRUZIONE DI SALTO CONDIZIONATO (TIPO SB)

```
beq x1, x2, offset
```

Salta (cioè cambia il valore del **Program Counter**) se `x1 == x2`. L'**offset** è relativo al **Program Counter attuale**.



COSA SUCCEDE DURANTE UNA BEQ

♦ 1. Lettura dei registri rs1 e rs2

- `x1` e `x2` vengono letti dal **register file** (come nel tipo R).

♦ 2. Confronto dei due registri

- Si usa la **ALU** per fare la **sottrazione**: $x1 - x2$
- La **Control Unit** imposta l'ALU per fare una **SUB**.
- Se il risultato è **0**, il bit **Zero = 1**, e il salto deve essere **eseguito (TAKEN)**.
- Altrimenti, **Zero = 0** e il salto **non viene eseguito**.



L'OFFSET

♦ Codifica

- È lungo 12 bit ma **sparso** in 4 posizioni nell'istruzione.
- Dopo essere ricostruito, viene:
 1. **Esteso con segno** a 32 bit.
 2. **Shiftato a sinistra di 1 bit** → perché i salti avvengono a indirizzi **pari** (quindi si moltiplica $\times 2$).
 - Esempio: offset `0b000000000100` → dopo lo shift `0b000000001000`.



CALCOLO DEL NUOVO PROGRAM COUNTER

Due possibilità:

- **Salto NON preso** → `PC = PC + 4`
- **Salto preso** → `PC = PC + offset`

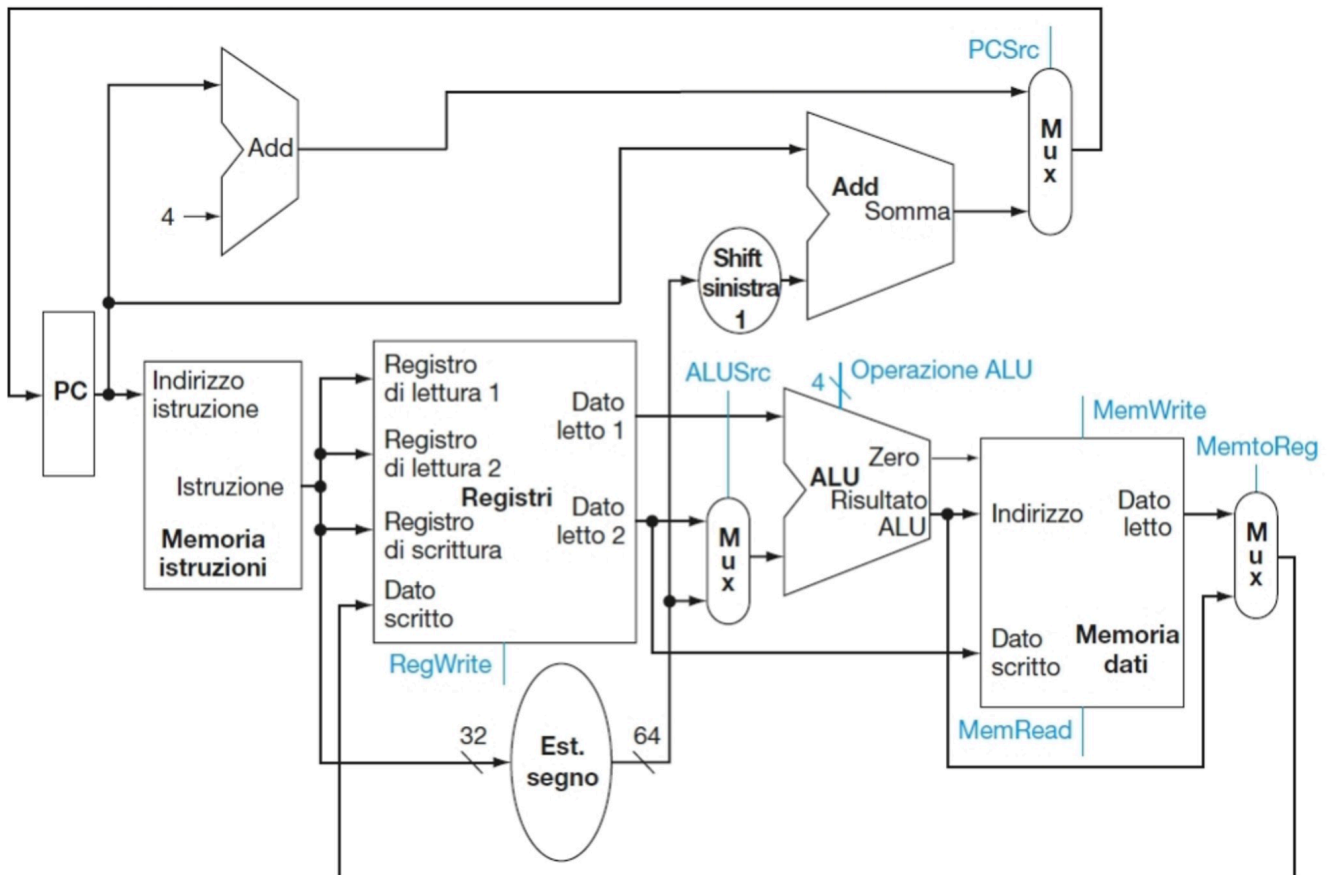
Il nuovo **PC** viene calcolato tramite un **MUX** (multiplexer):

La **Control Unit** usa il segnale **Zero** (dalla ALU) per **decidere quale input selezionare**:

- Se `Zero == 1` → **prende** `PC + offset`
- Se `Zero == 0` → **prende** `PC + 4`



UNIFICAZIONE: MOTIVO



Poiché le istruzioni hanno strutture diverse ma **devono usare gli stessi componenti hardware**, serve un'unica **unità di elaborazione** che possa:

1. Prendere i dati dai registri o dagli immediati.
2. Calcolare un risultato con la ALU.
3. Leggere/scrivere memoria se serve.
4. Aggiornare il Program Counter correttamente.

🔧 COMPONENTI CHIAVE E MULTIPLEXER

■ 1. Multiplexer prima della ALU

- Scopo: scegliere il **secondo operando della ALU**.
- **Scelte:**
 - Dato da **registro rs2** (istruzioni R-type).
 - **Immediato esteso con segno** (istruzioni I/S-type).
- 🧠 **Controllato da:** segnale `ALUSrc`, generato dalla Control Unit.

■ 2. Multiplexer prima del Register File (scrittura)

- Scopo: scegliere **cosa scrivere nel registro di destinazione**.
- **Scelte:**

- **Risultato della ALU** (es. `add`, `addi`, `ld` calcolo indirizzo).
- **Dato letto dalla memoria** (solo per `load`).
- 🧠 **Controllato da:** segnale `MemToReg`

■ 3. Multiplexer per aggiornare il PC

- Scopo: decidere il **nuovo valore del Program Counter**.
- **Scelte:**
 - `PC + 4` (istruzioni normali, default).
 - `PC + offset` (solo se salto `beq` è **TAKEN**).
- 🧠 **Controllato da:** segnale `PCSrc`.

🔑 SEGNALI DI CONTROLLO

🎯 ALUSrc

- Se `0` → secondo operando viene da `rs2` (R-type).
- Se `1` → viene dall'immediato esteso (I/S-type).
-

🎯 MemToReg

- Se `0` → scrivo nel registro il **risultato della ALU**.
- Se `1` → scrivo il **dato letto dalla memoria** (load).

🎯 RegWrite

- Se `1` → abilita la scrittura nel **register file**.

🎯 MemRead , MemWrite

- Attivano la **memoria dati** (per `ld` o `sd`).

🎯 PCSrc

- Se `0` → `PC + 4`
- Se `1` → `PC + offset` (quando eseguo un salto)

✳ Come si calcola `PCSrc` ?

$$\text{PCSrc} = \text{isBranch} \wedge \text{Zero} \quad \text{PCSrc} = \text{isBranch} \wedge \text{Zero}$$

- `isBranch` viene da `CODOP` (se è un'istruzione `beq`).

- Zero viene dalla ALU (se i due registri sono uguali).

FLUSSO GENERALE (semplificato)

1. **Decodifica istruzione** → identifica `rs1`, `rs2`, `rd`, `imm`.
2. **Leggi registri** da `rs1`, `rs2`.
3. **Immediato** → estensione del segno (se serve).
4. **ALU** esegue operazione (sottrazione per `beq`, somma per `add`, calcolo indirizzo per `load/store`).
5. **Memoria dati**: lettura/scrittura se necessario.
6. **MUX di scrittura** → decide cosa va in `rd`.
7. **MUX PC** → decide se il nuovo PC è `PC+4` o `PC+offset`.

Conclusione

Grazie all'aggiunta di **3 multiplexers** controllati dalla **Control Unit**, un'unica unità di elaborazione può eseguire correttamente:

- Istruzioni **aritmetiche** (tipo R),
- Istruzioni **load/store** (tipo I/S),
- Istruzioni di **salto condizionato** (tipo SB).

1. OBIETTIVO: UNIFICAZIONE DEL DATAPATH

Un unico datapath deve eseguire:

- **Istruzioni di tipo R** → `add`, `sub`
- **Istruzioni di tipo I/S** → `load`, `store`
- **Istruzioni di tipo SB** → `beq`

2. COMPONENTI AGGIUNTI PER L'UNIFICAZIONE

Per gestire tutti i tipi d'istruzione servono:

- **Multiplexer per ALUSrc** (seleziona secondo operando della ALU tra:
 - registro (tipo R)
 - immediato esteso (tipo I/S)
- **Multiplexer per MemToReg** (seleziona la sorgente del dato da scrivere nel `register file`):
 - da ALU (tipo R)

- da memoria (load)
 - **Multiplexer per PCSrc** (seleziona prossimo PC tra PC+4 e PC+offset):
 - controllato da: $PCSrc = Branch \text{ AND } Zero$
-



3. GESTIONE DELLE ISTRUZIONI

► Tipo R (add, sub)

- ALU: opera tra due registri
- Scrittura nel register file da ALU
- $ALUSrc = 0$, $MemToReg = 0$, $MemRead = 0$, $MemWrite = 0$, $RegWrite = 1$

► Tipo I/S (load, store)

- ALU calcola $indirizzo = base + offset$
- Load:
 - Legge dalla memoria → scrive nel registro
 - $MemRead = 1$, $MemWrite = 0$, $MemToReg = 1$, $RegWrite = 1$
- Store:
 - Scrive in memoria il valore del registro
 - $MemRead = 0$, $MemWrite = 1$, $RegWrite = 0$
- $ALUSrc = 1$ in entrambi

► Tipo SB (beq)

- ALU esegue: $rs1 - rs2$, controlla Zero
- Offset per salto calcolato con shift sinistro ($\times 2$)
- Somma offset con PC (ALU ADDER, parallela alla principale)
- Salto se: $Branch = 1 \text{ AND } Zero = 1$
- $ALUSrc = 0$, $MemRead/Write = 0$, $RegWrite = 0$



4. UNITÀ DI CONTROLLO PRINCIPALE

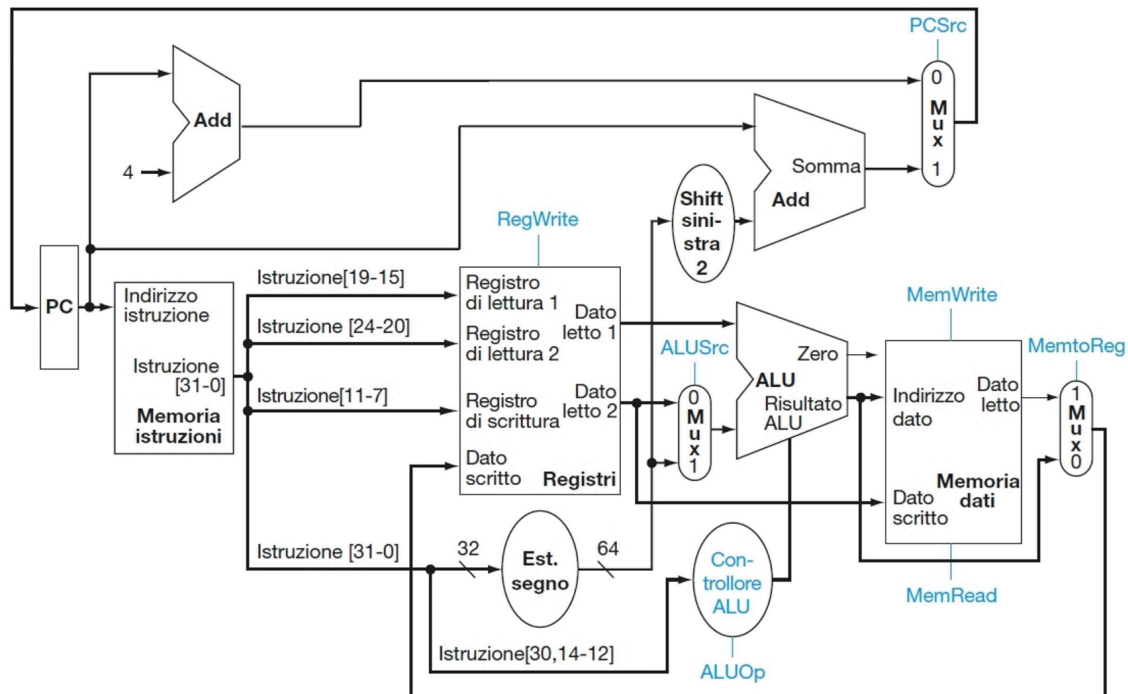
Ingressi:

- opcode (7 bit)

Uscite (segnali di controllo):

- RegWrite, MemWrite, MemRead, ALUSrc, MemToReg, Branch, ALUOp[1:0]

5. CONTROLLO DELLA ALU (ALU CONTROL UNIT)



Ingressi:

- ALUOp[1:0] dalla control unit principale
- funct3 (3 bit)
- funct7 (1 o più bit, es. per distinguere add/sub)

Output:

- 4 bit di controllo per la ALU

Tabella ALUOp:

ALUOp	Funzione	Istruzioni
00	Somma	Load, Store
01	Sottrazione	BEQ
10	Da funct3/7	Istruzioni R
11	Non usato	-

Segnale	Descrizione
MemWrite	Scrive nella memoria (1 per Store)
MemRead	Legge dalla memoria (1 per Load)
MemToReg	Seleziona tra memoria e ALU come sorgente per la scrittura nel registro
ALUSrc	Seleziona secondo operando ALU (registro o immediato)
Branch	Attiva salto (1 per beq)
PCSrc	Determina prossimo PC: PC+4 o PC+offset (calcolato come Branch AND Zero)

altro

Cos'è e come funziona il complemento a due? Quale problema risolve e perché è il più comodo?



Idea di base

Nel complemento a due:

- Il **bit più significativo** (cioè il primo a sinistra) ha **valore negativo**.
- Se il numero comincia con **0**, è positivo.
- Se comincia con **1**, è negativo.

Per esempio, in un sistema a **8 bit**:

- I numeri rappresentabili vanno da **-128 a +127**.
- In generale, con **n bit**, si rappresentano numeri da **-2^{n-1} a $2^{n-1} - 1$** .



Perché è comodo?

Il vantaggio principale è che, usando il complemento a due, **non servono circuiti separati per l'addizione e la sottrazione**.



I computer usano **lo stesso circuito (sommatore)** per entrambe le operazioni, senza controllare se i numeri sono negativi o positivi.

Questo semplifica:

- Il progetto dell'hardware
- I calcoli

- E riduce gli errori e i tempi di elaborazione

In pratica

Per ottenere il complemento a due (cioè la versione negativa di un numero binario):

1. **Inverto tutti i bit** del numero (complemento a uno)
2. **Aggiungo 1** al risultato

Esempio:

- +3 in binario su 4 bit: 0011
- Inverto: 1100
- Aggiungo 1 → 1101, che rappresenta -3

Conclusione

Il **complemento a due** è il sistema più efficiente per rappresentare numeri negativi in binario.

Grazie a lui, i computer possono **fare addizioni e sottrazioni con un solo circuito**, senza controllare il segno dei numeri.

È il metodo **standard** nei processori moderni perché è **preciso, semplice e veloce**.

Cos'è l'overflow?

L'**overflow** è un problema che si presenta nelle operazioni aritmetiche eseguite dal calcolatore, dovuto al fatto che i computer possono rappresentare solo numeri entro un certo intervallo, fissato dal numero di bit disponibili.

Quando si verifica l'overflow?

- L'overflow si verifica quando il risultato di un'operazione supera il **massimo valore rappresentabile** con i bit disponibili.
- Ad esempio, con n bit in complemento a due, i numeri rappresentabili sono tra $-2^{(n-1)}$ e $+2^{(n-1)} - 1$. Se il risultato esce da questo intervallo, si ha overflow.

Overflow in complemento a due

- Succede quando si sommano due numeri con **segno uguale** (entrambi positivi o entrambi negativi), ma il risultato ha un segno diverso.
- Oppure, a livello di bit, quando il riporto **dalla penultima colonna alla colonna più significativa** (bit di segno) e il riporto oltre il bit più significativo sono discordi (uno è 0 e l'altro è 1).

Come si gestisce l'overflow?

1. Senza trap

- L'hardware imposta un **bit di flag** in un registro speciale (es. il registro di stato).
- Il programma, se vuole, controlla questo flag dopo l'istruzione e, in caso di overflow, può eseguire una routine speciale (es. gestione errore).

2. Con trap

- Quando si verifica overflow, l'hardware **genera una trap** (interruzione sincrona).
- Il flusso di esecuzione passa a una procedura speciale (gestore della trap) che si occupa di gestire l'errore, ad esempio terminando il programma o correggendo l'errore.

Come funziona java (i vari passaggi per arrivare al linguaggio macchina)?

1. Scrittura del codice

Tutto parte dalla scrittura del programma in **Java**, che è un **linguaggio di alto livello**, leggibile da noi umani.

2. Compilazione

Quando compiliamo il codice (usando ad esempio `javac`), il compilatore **traduce il sorgente Java in un formato intermedio** chiamato **bytecode**.

 Il bytecode è contenuto nei file `.class` ed è **indipendente dal sistema operativo**.

3. Esecuzione tramite JVM

Il bytecode non viene eseguito direttamente dal processore, ma viene **interpretato dalla JVM**, cioè la **Java Virtual Machine**.

La JVM è un programma che:

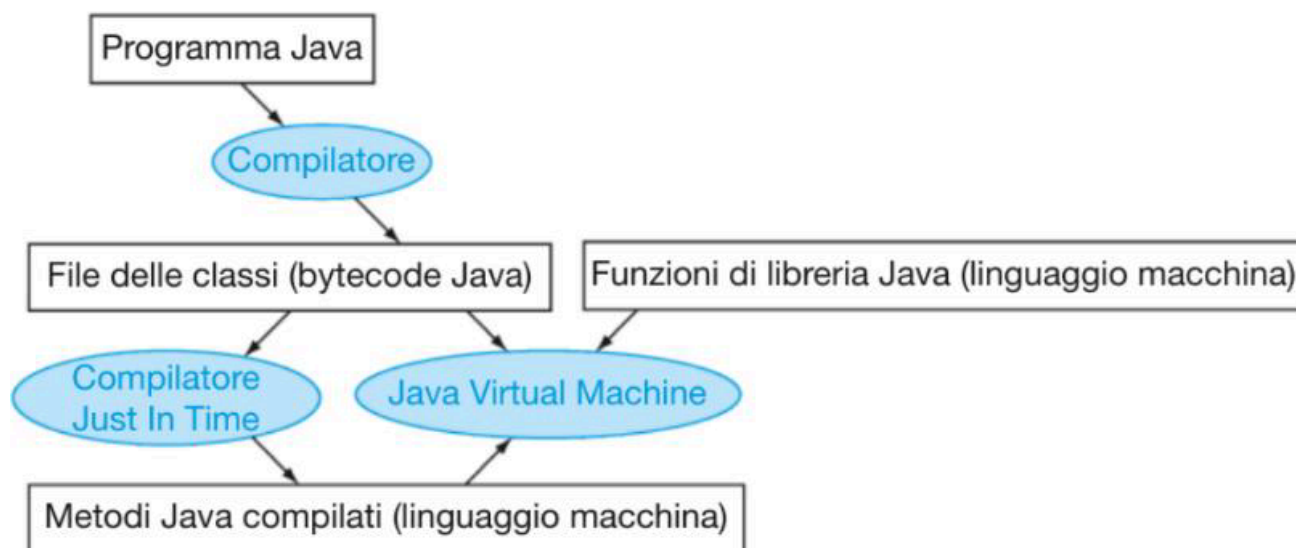
- legge il bytecode
- lo esegue su qualunque sistema (Windows, Linux, macOS, ecc.)
 - 👉 Questo è ciò che rende Java un **linguaggio portabile**: lo stesso programma funziona su più sistemi, basta avere una JVM.

4. Ottimizzazione con JIT

Durante l'esecuzione, la JVM può attivare un componente chiamato **JIT (Just-In-Time Compiler)**.

Il **JIT** traduce parti del bytecode in **linguaggio macchina nativo**, cioè quello comprensibile dal processore reale, per:

- migliorare le prestazioni
- evitare di interpretare ripetutamente lo stesso codice



✓ In sintesi

I passaggi principali sono:

1. Il codice Java viene compilato in **bytecode**
2. Il bytecode viene **interpretato dalla JVM**
3. La JVM può usare il **JIT** per tradurre il codice in **linguaggio macchina** nativo e renderlo più veloce

Questo sistema rende Java

- **portabile**
- **ottimizzato**
- e **sicuro** (perché il bytecode può essere controllato prima dell'esecuzione)

Quali sono i parametri per stabilire se un computer è più veloce?

per valutare la velocità di un computer, si usano diversi **parametri di prestazione**.

I principali sono:

🕒 1. Tempo di risposta (o tempo di esecuzione totale)

È il **tempo complessivo** necessario per completare un task. Comprende:

- tempo della CPU
- accessi alla memoria
- accessi al disco
- attività di input/output
- tempo richiesto dal sistema operativo

👉 In pratica: è il tempo che passa da quando do un comando a quando vedo il risultato.

2. Throughput (o larghezza di banda)

È il numero di programmi completati in un intervallo di tempo.

Esempio:

- Se un sistema completa 10 programmi in un minuto e un altro ne completa 20, il secondo ha un **throughput doppio**.

3. Tempo di esecuzione della CPU

È il tempo che la **CPU** passa effettivamente a elaborare un task (escludendo quindi I/O e sistema operativo).

Dipende da:

- **Numero di istruzioni** da eseguire
- **Cicli di clock per istruzione (CPI)**
- **Durata del ciclo di clock**
-  Questo si riassume nella formula:

 **formula**

Tempo CPU = (Numero istruzioni) × (Cicli/istruzione) × (Durata ciclo di clock)

4. Periodo di clock

È la **durata di un singolo ciclo di clock** (espresso in nanosecondi).

Un periodo più breve = clock più veloce = potenzialmente più prestazioni.